

SS2 DIGITAL TECHNOLOGIES LESSON NOTES

FIRST TERM

WEEK 1: REVIEW OF SS1 DIGITAL FOUNDATIONS

TOPIC: A COMPREHENSIVE REVIEW OF KEY SS1 CONCEPTS

INTRODUCTION: This week serves as a critical foundation for the advanced topics in SS2. We will conduct a deep review of the fundamental principles of digital technology, ensuring all students have the requisite knowledge to proceed. This is not a superficial overview; it is an in-depth re-examination of core concepts.

1: CORE COMPONENTS OF A COMPUTER SYSTEM

- **HARDWARE:** The tangible, physical components of the system.
 - **INPUT DEVICES:** (e.g., Keyboard, Mouse, Scanner, Microphone). These are the conduits for human-to-computer interaction, translating analog actions into digital signals.
 - **OUTPUT DEVICES:** (e.g., Monitor, Printer, Speakers). These translate digital signals back into human-readable (visual, auditory, or physical) formats.
 - **CENTRAL PROCESSING UNIT (CPU):** The "brain." It executes instructions. Its performance is measured in Gigahertz (GHz), representing billions of cycles per second. It consists of the Arithmetic Logic Unit (ALU) for calculations and the Control Unit (CU) for managing instructions.
 - **MEMORY (PRIMARY STORAGE):**
 - **RAM (RANDOM ACCESS MEMORY):** Volatile (temporary) memory used to hold data and applications *currently in use*. It is extremely fast and essential for multitasking.
 - **ROM (READ-ONLY MEMORY):** Non-volatile (permanent) memory that holds the **BIOS** (Basic Input/Output System) or **UEFI**, which is the firmware that boots the computer.
 - **STORAGE (SECONDARY STORAGE):**

- **HDD (HARD DISK DRIVE):** Non-volatile magnetic storage. Slower, but cheaper with high capacity.
- **SSD (SOLID-STATE DRIVE):** Non-volatile flash storage. Significantly faster than HDDs, more durable, but more expensive.
- **SOFTWARE:** The intangible set of instructions (programs) that tell the hardware what to do.
 - **SYSTEM SOFTWARE (OPERATING SYSTEM):** The master program that manages all other software and hardware. (e.g., Windows 11, macOS, Linux, Android). Its functions are:
 - Process management
 - Memory management
 - File system management
 - Hardware abstraction (I/O)
 - Providing a user interface (GUI/CLI)
 - **APPLICATION SOFTWARE:** Programs designed to perform specific tasks for the user. (e.g., MS Word, Google Chrome, Adobe Photoshop).

2: FUNDAMENTALS OF DATA REPRESENTATION

- **THE BINARY SYSTEM:** The native language of all digital computers. It uses only two digits: 0 (OFF) and 1 (ON).
- **BIT:** A single binary digit (a 0 or 1).
- **BYTE:** A group of 8 bits. A byte is the standard unit of measurement for digital data and can represent 256 (2^8) different values.
- **CHARACTER ENCODING:**
 - **ASCII (AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE):** An early 7-bit (later 8-bit) standard to map keyboard characters (A, b, !, 3) to numbers (which are then stored as bits). For example, 'A' is 65 (01000001).
 - **UNICODE (UTF-8):** A modern, vastly larger standard that includes characters and emojis from all global languages, ensuring universal compatibility.
- **IMAGE REPRESENTATION:**

- **PIXELS:** A digital image is a grid of "picture elements" or pixels.
- **COLOR DEPTH (RGB):** Each pixel's color is stored, often as a combination of Red, Green, and Blue values. A 24-bit "true color" system uses 8 bits (1 byte) for each, allowing for ~16.7 million colors.

3: NETWORKING AND INTERNET CONCEPTS

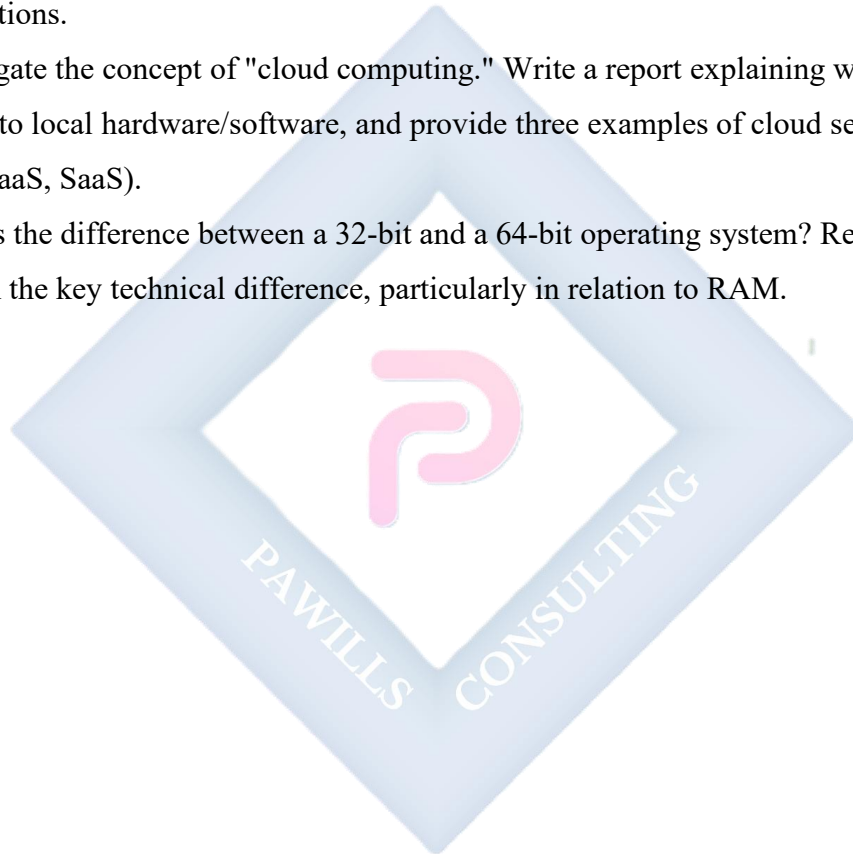
- **NETWORK TYPES:**
 - **LAN (LOCAL AREA NETWORK):** A network confined to a small geographical area (e.g., a home, school, or single office building).
 - **WAN (WIDE AREA NETWORK):** A network that spans a large geographical area (e.g., connecting a bank's Lagos and Abuja offices). The Internet is the largest WAN.
- **THE INTERNET:** A global "network of networks" that communicates using the TCP/IP (Transmission Control Protocol/Internet Protocol) suite.
- **IP ADDRESS:** A unique numerical label (e.g., 172.217.14.228) assigned to every device on a network. It acts like a digital street address.
- **DNS (DOMAIN NAME SYSTEM):** The "phonebook of the Internet." It translates human-readable domain names (e.g., www.google.com) into machine-readable IP addresses.

EVALUATION

1. In detail, differentiate between RAM and ROM, explaining *why* a computer needs both to function.
2. List and explain the four primary functions of an operating system.
3. Define "data representation" and explain the hierarchy from bit to byte. How many different values can a single byte represent, and why?
4. A student says, "My laptop's IP address is google.com." As an SS2 student, correct this statement with a detailed explanation of IP addresses, domain names, and the DNS.
5. Compare and contrast a Solid-State Drive (SSD) with a Hard Disk Drive (HDD) in terms of technology, speed, durability, and cost.

ASSIGNMENT

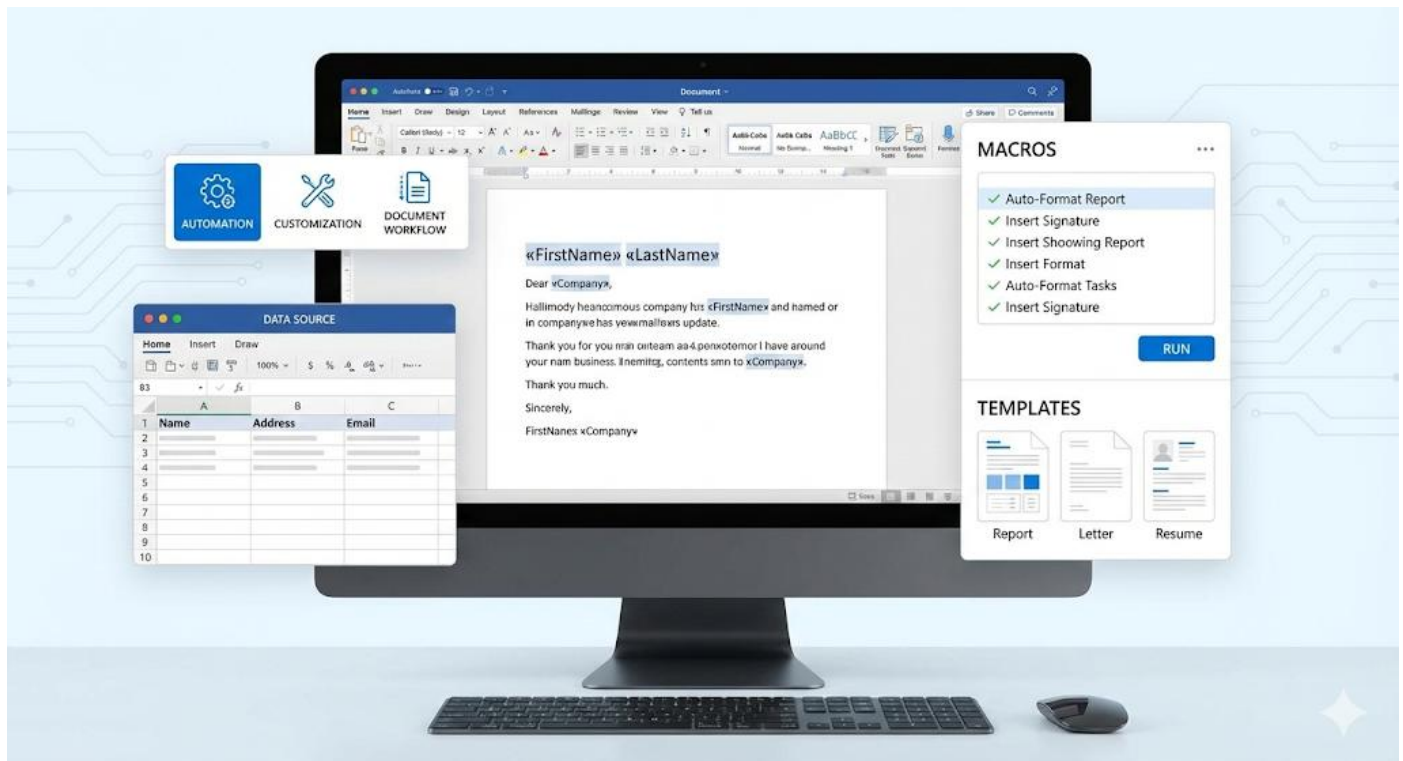
1. Write a 500-word essay on "The Evolution of the CPU," discussing the significance of Moore's Law and the shift from single-core to multi-core processors.
2. Research the ASCII and Unicode (UTF-8) character encoding standards. Create a table comparing them. Explain *why* the world transitioned from ASCII to Unicode.
3. Draw a detailed diagram of a typical home LAN. Your diagram must include a modem, a Wi-Fi router, a desktop (wired), a laptop (Wi-Fi), and a smartphone (Wi-Fi). Label all connections.
4. Investigate the concept of "cloud computing." Write a report explaining what it is, how it relates to local hardware/software, and provide three examples of cloud services (e.g., IaaS, PaaS, SaaS).
5. What is the difference between a 32-bit and a 64-bit operating system? Research and explain the key technical difference, particularly in relation to RAM.



WEEK 2: ADVANCED WORD PROCESSING TECHNIQUES

TOPIC: MAIL MERGE, MACROS, AND TEMPLATES

INTRODUCTION: This week, we transition from basic document formatting to powerful automation and professionalization tools within word processors. These skills are critical for



efficiency and consistency in academic and business environments.

1: MAIL MERGE IN-DEPTH

- **THE CONCEPT:** Mail merge is a process that automates the creation of personalized mass documents (such as letters, labels, or emails). It combines a static "main document" with a dynamic "data source" to produce a unique "merged document" for each recipient.
- **THE THREE COMPONENTS OF MAIL MERGE:**

1. **THE MAIN DOCUMENT (THE TEMPLATE):** This is the Word document containing the boilerplate text (the content that is the same for everyone) and special "merge fields" (e.g., <<First_Name>>) which act as placeholders for the dynamic data.
 2. **THE DATA SOURCE (THE RECIPIENT LIST):** This is the file that contains the individual information. It is typically an Excel spreadsheet, a Word table, an Outlook contact list, or a database file. The column headers (e.g., First_Name) must match the merge fields.
 3. **THE MERGED DOCUMENT (THE RESULT):** This is the final output, where the placeholders are replaced with the actual data from each row of the data source, creating a set of personalized documents.
- **THE STEP-BY-STEP PROCESS (USING THE MAIL MERGE WIZARD):**
 1. **START:** Go to the "Mailings" tab and select "Start Mail Merge" -> "Step-by-Step Mail Merge Wizard."
 2. **SELECT DOCUMENT TYPE:** Choose "Letters," "E-mail Messages," "Envelopes," etc.
 3. **SELECT STARTING DOCUMENT:** Choose "Use the current document."
 4. **SELECT RECIPIENTS:** Choose "Use an existing list" and browse to your data source (e.g., your Excel file).
 5. **WRITE YOUR LETTER:** This is where you insert the "merge fields." Click "Insert Merge Field" and add placeholders like <<Address>>, <<City>>, Dear <<Title>> <<Last_Name>>.
 6. **PREVIEW YOUR LETTERS:** You can scroll through each recipient's final letter.
 7. **COMPLETE THE MERGE:** You can "Print" directly or "Edit Individual Documents" to create a new, large Word file containing all the merged letters.

2: AUTOMATION WITH MACROS

- **THE CONCEPT:** A macro is a recorded sequence of commands, keystrokes, and actions that can be saved and executed (played back) with a single click or keyboard shortcut.

- **THE PURPOSE:** To automate highly repetitive and complex tasks, saving time and reducing human error.
- **HOW TO RECORD A MACRO (NO CODING REQUIRED):**
 1. **ENABLE THE DEVELOPER TAB:** (File -> Options -> Customize Ribbon -> Check "Developer").
 2. **START RECORDING:** Go to the "Developer" tab and click "Record Macro."
 3. **ASSIGN:** Give the macro a name (no spaces) and assign it to a button or keyboard shortcut (e.g., Alt+C).
 4. **PERFORM ACTIONS:** Every single click and keystroke is now being recorded.
For example:
 - Change font to "Times New Roman."
 - Change font size to "12."
 - Set line spacing to "Double."
 - Apply a first-line indent.
 5. **STOP RECORDING:** Go to the "Developer" tab and click "Stop Recording."
- **MACRO SECURITY (CRITICAL):**
 - Macros are powerful pieces of code (VBA - Visual Basic for Applications).
 - **MACRO VIRUSES** are malicious programs embedded in documents. When the document is opened and macros are enabled, the virus can execute, damaging files or stealing data.
 - **RULE:** *Never* enable macros from an untrusted source or an unexpected email attachment.

3: PROFESSIONAL DOCUMENT TEMPLATES

- **THE CONCEPT:** A template is a pre-designed "blueprint" or "mold" for a document. It is not a regular document.
- **FILE EXTENSION:** A standard Word document is .docx. A Word template is .dotx.
- **THE DIFFERENCE:** When you "open" a .docx, you are editing the original file. When you "open" a .dotx, Word automatically creates a *new, untitled document* based on the template, leaving the original template unchanged.

- **WHY USE TEMPLATES?**

1. **CONSISTENCY:** Ensures all company documents (letters, memos, reports) have the same logo, font, and layout.
2. **BRANDING:** Reinforces a professional brand identity.
3. **EFFICIENCY:** Saves time by not having to recreate the layout for every new document.

- **CREATING A CUSTOM TEMPLATE:**

1. Design your document (e.g., create a company letterhead and footer).
2. Go to "File" -> "Save As."
3. Change the "Save as type" dropdown from "Word Document (.docx)" to *"Word Template (.dotx)."*
4. Save the file. It will now appear in your "Personal Templates" section.

EVALUATION

1. Define "Mail Merge" and list its three core components with a brief description of each.
2. What is a macro? Explain the primary benefit of using one and the single greatest risk associated with them.
3. Explain the functional difference between a .docx file and a .dotx file. How does the software treat them differently upon opening?
4. During a mail merge, what is the purpose of a "merge field" (e.g., <<First_Name>>) in the main document?
5. Describe a real-world scenario in a school or business (other than a letter) where a macro could be used to save a significant amount of time.

ASSIGNMENT

1. **PRACTICAL (MAIL MERGE):** Create an Excel file named DataSource.xlsx with 10 fictional names, addresses, and an "Outstanding_Fee." Then, create a Word document WarningLetter.docx and use Mail Merge to write a personalized letter to each person, addressing them by name and stating their specific outstanding fee. Submit the merged document.

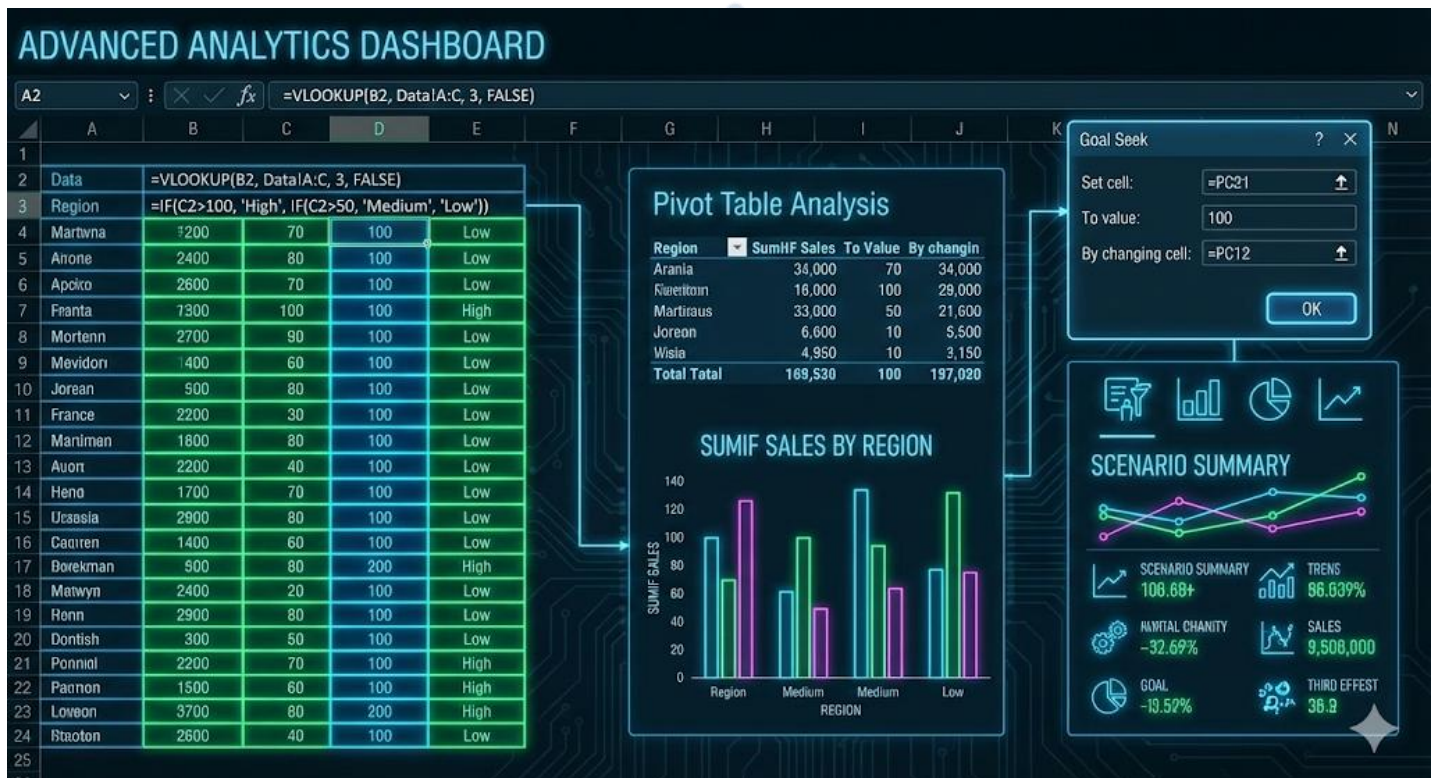
2. **PRACTICAL (MACRO):** Create and record a macro assigned to the shortcut Alt+H.
The macro must perform the following four actions: (1) change the font to "Arial," (2) set the font size to "18," (3) bold the text, and (4) center the text. Write a paragraph and test your macro on it.
3. **PRACTICAL (TEMPLATE):** Design a professional, one-page resume (Curriculum Vitae) for a fictional SS2 student. Pay attention to layout, headings, and consistency. Save this file as ResumeTemplate.dotx.
4. **RESEARCH:** Investigate the VBA (Visual Basic for Applications) code behind a simple macro. Record a macro, then go to "Developer" -> "Macros" -> "Edit." Copy and paste the code and write a short explanation of what you think the code is doing.
5. **CRITICAL THINKING:** Your boss asks you to send a personalized holiday greeting e-mail to 1,000 clients. Would you use Mail Merge for this? What are the advantages and potential disadvantages of using Mail Merge for e-mail blasts?



WEEK 3: ADVANCED SPREADSHEET ANALYSIS I

TOPIC: PIVOT TABLES AND ADVANCED FORMULAS

INTRODUCTION: This week, we move beyond basic spreadsheet calculations (=SUM, =AVERAGE) and into the realm of dynamic data analysis. Pivot tables are arguably the most powerful data summarization tool in Excel, while logical and lookup functions allow you to



build "smart" spreadsheets that react to your data.

1: MASTERING PIVOT TABLES

- **THE CONCEPT:** A pivot table is an interactive data summarization tool that allows you to quickly aggregate, analyze, and "pivot" (rearrange) large datasets without using any formulas.

- **THE PROBLEM PIVOT TABLES SOLVE:** They transform a large, "flat" table (e.g., 10,000 rows of sales data) into a dynamic and concise summary report (e.g., "Total sales by region and product category").
- **SOURCE DATA REQUIREMENTS:** Your raw data *must* be "clean" and in a tabular format:
 1. Each column must have a unique header (e.g., "Date," "Salesperson," "Amount").
 2. There must be no empty rows or columns within the data.
- **THE PIVOT TABLE INTERFACE:**
 - **FIELDS LIST:** A list of all your column headers.
 - **ROWS AREA:** Dragging a field here (e.g., "Region") creates a unique row for each item in that field.
 - **COLUMNS AREA:** Dragging a field here (e.g., "Product") creates a unique column for each item.
 - **VALUES AREA:** The "calculation engine." Dragging a numerical field here (e.g., "Amount") defaults to Sum. Dragging a text field defaults to Count.
 - **FILTERS AREA:** Used to filter the entire report by a specific field.
- **CREATING A PIVOT TABLE (STEP-BY-STEP):**
 0. Click anywhere inside your clean data.
 1. Go to the "Insert" tab -> "PivotTable." Excel will auto-select your data.
 2. Choose "New Worksheet" and click "OK."
 3. Drag and drop your fields. **Example:**
 - Drag "Region" to **ROWS**.
 - Drag "Product" to **COLUMNS**.
 - Drag "Amount" to **VALUES**.
 4. You now have an instant summary table. You can "pivot" the data by dragging "Region" to Columns and "Product" to Rows.
- **MANIPULATING PIVOT TABLES:**
 - **REFRESHING:** Pivot tables do *not* update automatically. If you change the source data, you must right-click the pivot table and select "Refresh."

- **GROUPING:** You can group dates (into months/quarters), numbers (into ranges), etc.

2: ADVANCED FORMULAS AND FUNCTIONS

• LOGICAL FUNCTIONS (THE IF STATEMENT):

- **PURPOSE:** To make your spreadsheet return different results based on a condition.
- **SYNTAX:** =IF(logical_test, value_if_true, value_if_false)
- **EXAMPLE:** =IF(A2 >= 50, "Pass", "Fail")
 - logical_test: Is the value in cell A2 greater than or equal to 50?
 - value_if_true: If yes, display "Pass."
 - value_if_false: If no, display "Fail."

• NESTED IF STATEMENTS:

- **PURPOSE:** To handle multiple conditions by placing an IF statement inside another IF statement.
- **EXAMPLE (GRADING):** =IF(A2 >= 70, "A", IF(A2 >= 60, "B", IF(A2 >= 50, "C", "F")))

• LOOKUP FUNCTIONS (VLOOKUP):

- **PURPOSE:** To "vertically lookup" a value in one table and return a corresponding value from another column.
- **SYNTAX:** =VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])
- **BREAKDOWN OF ARGUMENTS:**
 1. lookup_value: *What* you are looking for (e.g., a student's ID in cell A2).
 2. table_array: *Where* to look. This is the lookup table (e.g., a separate table E1:F50 containing all student IDs and their names). **CRITICAL:** The lookup value *must* be in the first column of this table.
 3. col_index_num: *Which* column's value to return from the table_array. (e.g., 2 to return the student's name from the 2nd column).
 4. [range_lookup]:

- FALSE (or 0): Exact Match. (99% of the time, you want this).
- TRUE (or 1): Approximate Match. (Used for tax brackets, etc.).
- **CONDITIONAL AGGREGATION (SUMIF, COUNTIF, AVERAGEIF):**
 - **PURPOSE:** To sum, count, or average cells *only if* they meet a certain condition.
 - **SYNTAX:** =SUMIF(range, criteria, [sum_range])
 - **EXAMPLE:** =SUMIF(A1:A100, "Lagos", B1:B100)
 - This will look in the range A1:A100 for the text "Lagos" and will *only* sum the corresponding values from B1:B100.

EVALUATION

1. Define a pivot table and explain the two most important requirements for your source data.
2. What are the four main areas of the pivot table interface, and what is the function of the "Values" area?
3. Write the full syntax for a VLOOKUP function and explain what *each* of its four arguments represents.
4. What is the purpose of the SUMIF function? Write a formula to calculate the total sales (in column C) for only the "Electricals" category (in column B).
5. Write a nested IF statement for cell B1 that checks the score in cell A1. It should return "Distinction" for scores > 85 and "Pass" for scores > 50. If neither, it should return "Retake."

ASSIGNMENT

1. **PRACTICAL (DATASET):** Create a 20-row Excel sheet of sales data with 5 columns: Date, Region (use "North," "South," "East," "West"), Salesperson (use 5 names), Category ("Electronics," "Furniture"), and Amount.
2. **PRACTICAL (PIVOT 1):** Using the dataset from Q1, create a pivot table on a new sheet that shows the total Amount (Sum) for each Region (as rows).
3. **PRACTICAL (PIVOT 2):** Using the dataset from Q1, create a second pivot table that shows the total Amount for each Salesperson (rows) broken down by Category (columns).

4. **PRACTICAL (VLOOKUP):** On a new sheet, create a small table (the "lookup table") with two columns: Salesperson (list the 5 names) and Commission_Rate (assign a different % to each). Go back to your main dataset and add a 6th column. In this new column, use VLOOKUP to automatically pull the correct commission rate for the salesperson in that row.
5. **PRACTICAL (SUMIF):** At the bottom of your main dataset, create a small summary section. Use the SUMIF function to calculate the total sales for "Electronics" and a separate SUMIF for the total sales for "Furniture."



WEEK 4: ADVANCED SPREADSHEET ANALYSIS II

TOPIC: FINANCIAL MODELS AND SCENARIO ANALYSIS

INTRODUCTION: This week, we leverage the power of spreadsheets to move from *analyzing the past* (pivot tables) to *forecasting the future*. We will learn how to build basic financial models and use "what-if" analysis tools to make informed business decisions.



1: INTRODUCTION TO FINANCIAL MODELING

- **THE CONCEPT:** A financial model is a spreadsheet-based simulation of a real-world financial situation. It uses a set of inputs (assumptions) to calculate a set of outputs (forecasts), allowing a user to see how a change in an assumption affects the outcome.
- **PURPOSE:**
 1. **FORECASTING:** Predicting future revenue, profit, or cash flow.
 2. **BUDGETING:** Planning for a company's or project's expenses.

3. **DECISION MAKING:** Determining the financial viability of a new project (e.g., "Should we open a new branch?").
- **KEY PRINCIPLES OF GOOD MODELING:**
 1. **ASSUMPTIONS-DRIVEN:** All key inputs (e.g., "Price," "Sales_Growth_Rate") should be in separate, clearly labeled cells. **Never** "hard-code" a number (e.g., $A1*1.05$) inside a formula. Instead, use $A1 * B1$, where B1 is a cell labeled "Growth_Rate."
 2. **FLEXIBLE:** The model should be easy to update. Changing an assumption in one cell should cause the entire model to recalculate.
 3. **CLEAR AND SIMPLE:** A good model is easy to understand, with clear labels and formatting.
 - **BASIC EXAMPLE: A SIMPLE PROFIT/LOSS FORECAST:**
 - **ASSUMPTIONS SECTION:**
 - Price per Unit = N1,000
 - Cost per Unit = N400
 - Units Sold = 500
 - **CALCULATIONS SECTION (THE MODEL):**
 - Revenue = Price per Unit * Units Sold (=N500,000)
 - Cost of Goods Sold (COGS) = Cost per Unit * Units Sold (=N200,000)
 - Gross Profit = Revenue - COGS (=N300,000)

2: SCENARIO ANALYSIS & "WHAT-IF"

- **THE CONCEPT:** Scenario analysis is the process of testing how a model's outputs (e.g., "Profit") change when one or more of the inputs (assumptions) are changed.
- **THE "WHAT-IF" QUESTION:** This answers questions like, "What if our units sold are only 300?" or "What if the cost per unit increases to N500?"
- **USING THE 'SCENARIO MANAGER' TOOL:**
 - Instead of manually changing the assumption cells, Scenario Manager allows you to save different sets of assumptions as presets.
 - **LOCATION:** "Data" tab -> "What-If Analysis" -> "Scenario Manager."

- **PROCESS:**

1. Click "Add" to create a scenario (e.g., "Best Case").
2. Select the "Changing cells" (your assumption cells, e.g., Price per Unit and Units Sold).
3. Enter the values for that scenario (e.g., Price=N1,200, Units=700).
4. Repeat for "Worst Case" (e.g., Price=N900, Units=300).
5. You can now instantly toggle between these scenarios to see the profit change.

3: THE 'GOAL SEEK' TOOL

- **THE CONCEPT:** Goal Seek is a "what-if" analysis tool that works *backwards*. You specify the *desired result* (the "goal"), and Goal Seek finds the *input value* required to achieve it.
- **THE PROBLEM IT SOLVES:** It answers questions like, "How many units must I sell to achieve a profit of N500,000?" or "What price must I charge to break even (profit = N0)?"
- **HOW TO USE GOAL SEEK:**
 - **LOCATION:** "Data" tab -> "What-If Analysis" -> "Goal Seek."
 - It has three simple fields:
 1. **SET CELL:** The cell containing your final formula (e.g., the "Gross Profit" cell).
 2. **TO VALUE:** The numerical goal you want to achieve (e.g., 500000).
 3. **BY CHANGING CELL:** The *one* assumption cell you want Excel to change (e.g., the "Units Sold" cell).
 - Click "OK," and Excel will run iterations until it finds the answer.

EVALUATION

1. Define "financial model" and state its three primary purposes.
2. What is the most important principle of good modeling? (Hint: It involves "hard-coding").

3. Explain the difference between "Scenario Manager" and "Goal Seek." Which one works backwards?
4. Provide a practical, real-world question that "Goal Seek" could be used to answer.
5. What are the three parameters (fields) you must provide when using the Goal Seek tool?

ASSIGNMENT

1. **PRACTICAL (MODEL):** Build a simple financial model for a small business (e.g., a bakery).
 - **ASSUMPTIONS:** Price per Loaf, Cost of Ingredients per Loaf, Monthly Shop Rent, Monthly Staff Salary.
 - **INPUT:** Loaves Sold per Month.
 - **CALCULATIONS:** Total Revenue, Total Cost of Loaves, Total Fixed Costs (RENT+SALARY), NET PROFIT.
2. **PRACTICAL (SCENARIO MANAGER):** Using the model from Q1, create three scenarios using Scenario Manager:
 - "Worst Case": (Low Price per Loaf and low Loaves Sold).
 - "Expected Case": (Your original assumptions).
 - "Best Case": (High Price per Loaf and high Loaves Sold).
3. **PRACTICAL (GOAL SEEK):** Using the model from Q1, use Goal Seek to determine the *exact* number of Loaves Sold per Month required to achieve a NET PROFIT of N100,000.
4. **PRACTICAL (GOAL SEEK 2):** Using the model from Q1, use Goal Seek again to determine the Price per Loaf you would need to charge to "break even" (NET PROFIT = 0), assuming you only sell your "Worst Case" number of loaves.
5. **RESEARCH:** Investigate the "Solver" add-in in Excel. How is it similar to, and more powerful than, Goal Seek? (Hint: Constraints).

WEEK 5: MIDTERM TEST

TOPIC: ASSESSMENT OF WEEKS 1-4

INTRODUCTION: This week is dedicated to a formal assessment of all concepts covered in this term, from the SS1 review to advanced spreadsheet analysis.

SCOPE:

- Week 1: Hardware, software, OS, data representation, networking.
- Week 2: Mail merge, macros, templates.
- Week 3: Pivot tables, IF, nested IF, VLOOKUP, SUMIF.
- Week 4: Financial modeling, Scenario Manager, Goal Seek.

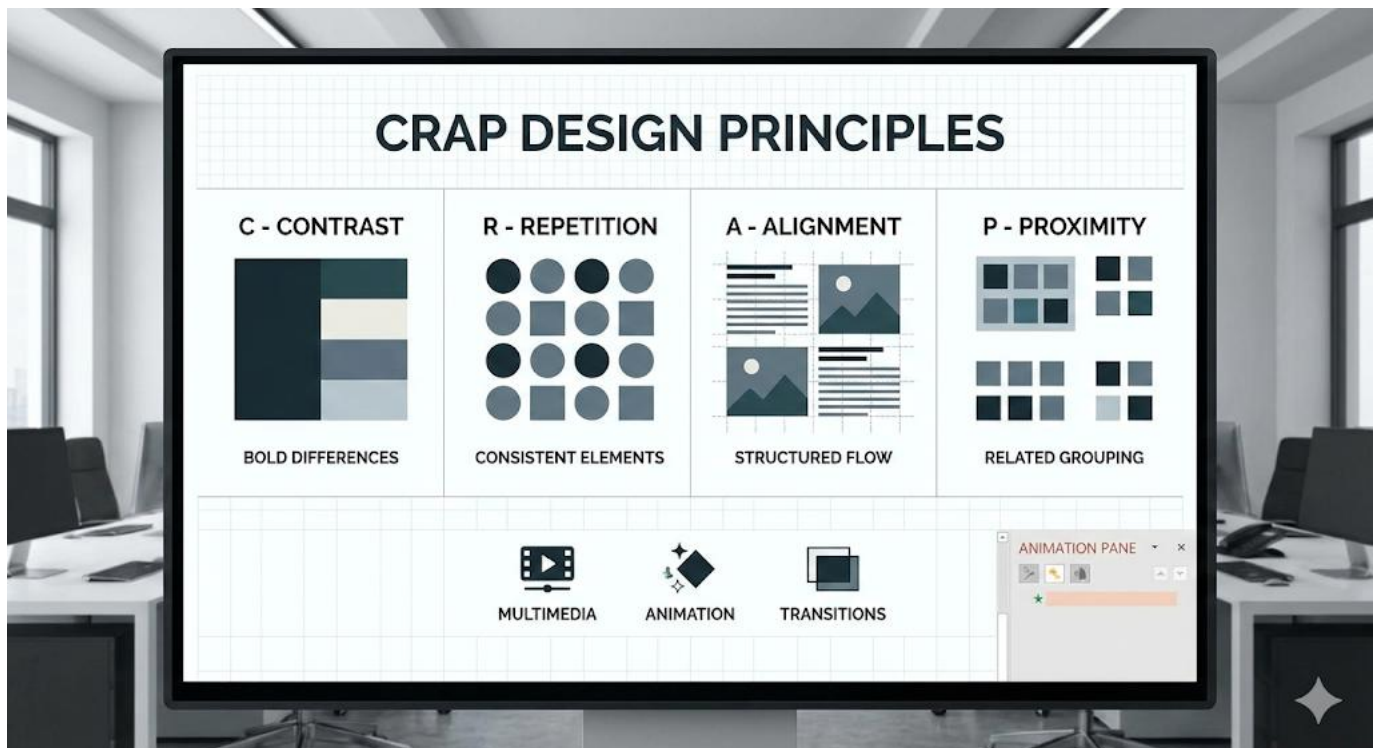
FORMAT:

- **SECTION A: MULTIPLE CHOICE QUESTIONS (20 MARKS)** - Testing definitions and concepts.
- **SECTION B: SHORT ANSWER QUESTIONS (30 MARKS)** - Testing explanations (e.g., "Explain the 3 components of Mail Merge").
- **SECTION C: PRACTICAL APPLICATION/THEORY (50 MARKS)** - (e.g., "Write the VLOOKUP formula for..." or "Describe the steps to create a pivot table...").

WEEK 6: ADVANCED PRESENTATION DESIGN AND DELIVERY

TOPIC: MULTIMEDIA INTEGRATION AND PROFESSIONAL DESIGN

INTRODUCTION: Most people use presentation software (like PowerPoint) poorly, creating boring, text-heavy slides. This is known as "Death by PowerPoint." This week, we learn advanced techniques to create dynamic, professional, and truly engaging multimedia



presentations.

1: PROFESSIONAL DESIGN PRINCIPLES (BEYOND BULLET POINTS)

- **THE C.R.A.P. PRINCIPLES OF DESIGN:** A simple framework for professional layout.
 1. **CONTRAST:** Make different elements (e.g., headline and text) *really* different. Avoid similar fonts or colors. This creates visual interest.

2. **REPETITION:** Repeat key elements (fonts, colors, logos) throughout the presentation. This creates a sense of unity and branding.
 3. **ALIGNMENT:** Place every element intentionally. Align text and images to a grid. Avoid "floating" elements. This creates an orderly, professional look.
 4. **PROXIMITY:** Group related items close together. This creates logical sections and reduces clutter.
- **USING THE 'SLIDE MASTER' (THE MOST IMPORTANT EFFICIENCY TOOL):**
 - **LOCATION:** "View" tab -> "Slide Master."
 - **CONCEPT:** The Slide Master is *not* a slide in your presentation. It is the "template" that controls *all* the slides.
 - **PURPOSE:** If you want your company logo, a page number, or a specific font on *every single slide*, you add it **once** to the Slide Master. This ensures 100% consistency and saves you from copy-pasting.

2: MULTIMEDIA INTEGRATION

- **INSERTING AND FORMATTING VIDEO:**
 - **EMBEDDING VS. LINKING:**
 - **LINKING:** The video file remains separate. **RISK:** If you move the presentation file without the video file, the link will break.
 - **EMBEDDING (RECOMMENDED):** The video file is saved *inside* the presentation file. This makes the file size larger, but it is safer and more portable.
 - **VIDEO TOOLS:** Once inserted, the "Playback" tab appears. You can:
 - Trim the video (cut the start and end).
 - Set it to "Play Automatically" or "On Click."
 - Set it to "Loop Until Stopped."
- **INSERTING AND FORMATTING AUDIO:**
 - **NARRATION:** You can record your own voice explaining each slide ("Record Slide Show").

- **BACKGROUND MUSIC:** You can add a music file, set it to "Play in Background," "Play Across Slides," and "Loop."

3: ADVANCED ANIMATIONS AND TRANSITIONS

- **THE CRITICAL DIFFERENCE:**

- **TRANSITIONS:** Affect how the entire *slide* enters or leaves the screen (e.g., Fade, Push).
- **ANIMATIONS:** Affect individual *objects* (text, images) *on* a slide (e.g., Appear, Fly In).

- **EFFECTIVE USE VS. POOR USE:**

- **POOR:** Using distracting animations (like "Spin" or "Bounce") on every bullet point.
- **EFFECTIVE:** Using subtle animations (like "Appear" or "Fade") to reveal information sequentially, allowing the presenter to control the pace.

- **THE ANIMATION PANE:**

- **LOCATION:** "Animations" tab -> "Animation Pane."
- **PURPOSE:** A timeline that gives you precise control over your animations. You can reorder them, set them to play "With Previous" or "After Previous," and adjust their timing.

- **THE 'MORPH' TRANSITION (A MODERN POWERPOINT TOOL):**

- Morph is a transition that *looks* like an animation.
- If you duplicate a slide and then move or resize an object on the second slide, applying "Morph" will make PowerPoint *automatically animate* the object's movement from its first position to its second.

4: INTERACTIVE PRESENTATIONS

- **NON-LINEAR PRESENTATIONS:** Most presentations are linear (Slide 1 -> 2 -> 3). A non-linear presentation allows the user to choose their path, like a website.
- **HOW TO CREATE:**

1. **HYPERLINKS:** You can hyperlink any object (text, shape, image) to another slide, a website, or a file.
2. **ACTION BUTTONS:** (Insert -> Shapes -> Action Buttons). These are pre-made buttons for "Home," "Next," "Previous."
3. **MAIN MENU:** Create a "Main Menu" or "Table of Contents" slide, and make each agenda item a hyperlink that jumps to that specific section.

EVALUATION

1. Explain the purpose of the "Slide Master" and provide two examples of its use.
2. What is the critical difference between an "Animation" and a "Transition"?
3. Briefly define the four C.R.A.P. principles of design.
4. What is the difference between *embedding* a video and *linking* to a video? Which is safer for portability and why?
5. Define a "non-linear presentation" and list the two main tools (hyperlinks, action buttons) used to create one.

ASSIGNMENT

1. **PRACTICAL (SLIDE MASTER):** Create a 5-slide presentation on "The History of Nigeria." Use the Slide Master to add a Nigerian flag logo to the top-right corner and a page number to the bottom-left corner of *every* slide.
2. **PRACTICAL (MULTIMEDIA):** On one slide of your presentation, embed a short (school-appropriate) YouTube video related to your topic and set it to play "On Click."
3. **PRACTICAL (ANIMATION):** On another slide, create a list of 4 bullet points. Use the "Animation Pane" to make each point "Appear" one by one, *only* when you click.
4. **PRACTICAL (INTERACTIVITY):** Create a "Main Menu" on Slide 1. Create two shapes (buttons) labeled "History" and "Culture." Make "History" a hyperlink that jumps to Slide 2, and "Culture" a hyperlink that jumps to Slide 4.
5. **RESEARCH:** Find a poorly designed presentation online (or a template you dislike). Write a 300-word critique of it based on the C.R.A.P. principles. Explain how you would improve it.

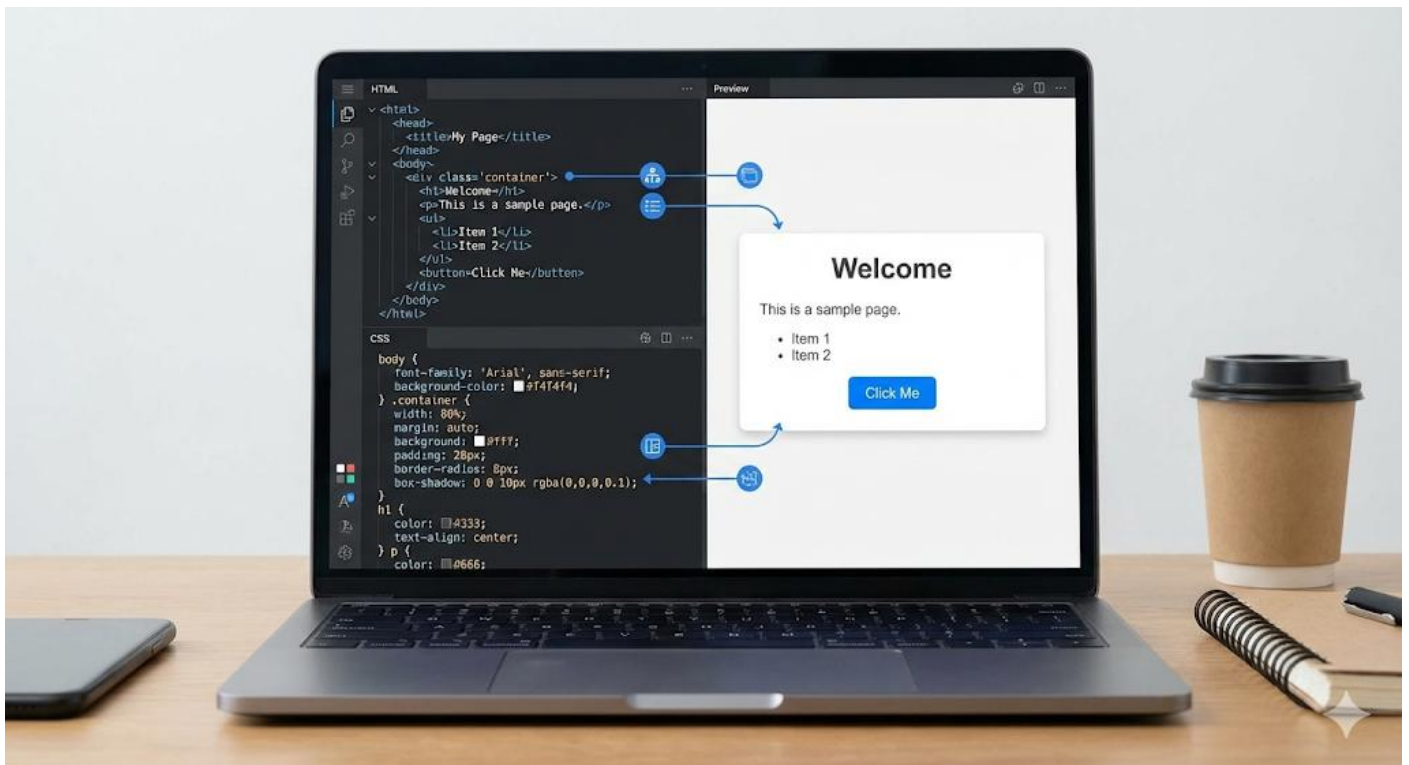
WEEK 7: MIDTERM BREAK



WEEK 8: WEB DESIGN I - INTRODUCTION TO HTML AND CSS

TOPIC: THE FUNDAMENTALS OF WEB STRUCTURE AND STYLE

INTRODUCTION: This week, we begin one of the most important topics in Digital Technology: Web Design. We will learn the two foundational languages of the entire World



Wide Web: HTML (for structure) and CSS (for style).

1: THE CLIENT-SERVER MODEL

- **WORLD WIDE WEB (WWW):** A system of interlinked documents (websites) accessed via the Internet.
- **THE MODEL:**
 1. **CLIENT:** The user's web browser (e.g., Google Chrome, Safari) on their device (laptop, phone).

2. **REQUEST:** The client "requests" a website (e.g., by typing a URL).
3. **SERVER:** A powerful, permanently-connected computer (a "web server") that "hosts" (stores) the website's files (HTML, CSS, images).
4. **RESPONSE:** The server finds the requested files and "serves" (sends) them back to the client's browser, which then assembles and displays the webpage.

2: HTML - THE SKELETON OF THE WEB

- **WHAT IS HTML?** Hypertext Markup Language.
 - **HYPERTEXT:** Text that contains links (hyperlinks) to other texts.
 - **MARKUP LANGUAGE:** A language that uses "tags" to *describe* or *structure* the content. It is **not** a programming language; it has no logic.
- **THE CONCEPT OF 'TAGS':**
 - Tags are keywords in angle brackets: <tagname>.
 - They almost always come in pairs: an opening tag <p> and a closing tag </p>.
 - <p>This is a paragraph.</p> (The tags "mark up" the content).
- **THE BASIC HTML DOCUMENT STRUCTURE (THE "BOILERPLATE"):**
- <!DOCTYPE html>
- <html>
- <head>
- <title>This is the Page Title (shows in browser tab)</title>
- </head>
- <body>
- <!-- All visible content goes here -->
- <h1>This is a main heading.</h1>
- <p>This is a paragraph.</p>
- </body>
- </html>
- **ESSENTIAL HTML TAGS:**
 - <h1> to <h6>: Heading tags (H1 is the most important, H6 is the least).
 - <p>: Paragraph.
 - : Anchor (link) tag. href is an "attribute."

- Example: `<a href="https://www.google.com">Click to visit Google</a>`
- ``: Image tag (self-closing).
 - `src`: The "source" (path or URL) of the image.
 - `alt`: "Alternate text" (critical for accessibility/screen readers and if the image breaks).
- `<ul>` (Unordered List) & `<ol>` (Ordered List): Used with `<li>` (List Item) tags.

3: CSS - THE SKIN OF THE WEB

- **WHAT IS CSS?** Cascading Style Sheets.
- **THE PURPOSE:** CSS controls the entire *presentation, style, and layout* of the HTML content.
- **SEPARATION OF CONCERNS (THE "WHY"):** HTML is for *content/structure*. CSS is for *style*. They should be kept in separate files. This makes websites easier to maintain.
- **THREE WAYS TO ADD CSS (FROM BEST TO WORST):**
 1. **EXTERNAL (BEST PRACTICE):** A separate .css file (e.g., style.css). It is linked in the HTML `<head>`:
 - `<link rel="stylesheet" href="style.css">`
 2. **INTERNAL (OKAY FOR TESTING):** CSS code is placed inside a `<style>` tag in the HTML `<head>`.
 3. **INLINE (BAD PRACTICE):** CSS is placed directly inside an HTML tag using the `style` attribute:
 - `<p style="color: red; font-size: 20px;">`
- **BASIC CSS SYNTAX (SELECTOR, PROPERTY, VALUE):**
 - `/* This is a CSS comment */`
 -
 - `selector {`
 - `property: value;`
 - `}`
 -
 - `h1 {`

- color: blue;
- font-size: 32px;
- }
-
- p {
- color: #333333; /* A dark grey hex code */
- font-family: Arial, sans-serif;
- }

EVALUATION

1. In your own words, describe the client-server model of the World Wide Web.
2. What does HTML stand for? Explain *why* it is called a "markup language" and not a "programming language."
3. Write the full, basic "boilerplate" skeleton of an HTML5 document, including the DOCTYPE, html, head, title, and body tags.
4. Write the full HTML code for an image that: (1) links to images/logo.png and (2) has an alt text of "Company Logo."
5. Explain the three methods of adding CSS to a document (External, Internal, Inline). Which is the best practice and *why* (Hint: "Separation of Concerns")?

ASSIGNMENT

1. **PRACTICAL (HTML):** Create a file named index.html. Create a simple "About Me" webpage. It *must* include:
 - One <h1> tag (Your name).
 - One <h2> tag ("My Hobbies").
 - One tag (Find a URL for a hobby-related image and use it).
 - One <p> tag (A short paragraph about yourself).
 - One (unordered list) of at least three hobbies.
2. **PRACTICAL (HTML LINKS):** On your index.html page, add a link to your favorite website.

3. **PRACTICAL (CSS):** Create a new file named style.css in the *same folder* as your index.html.
4. **PRACTICAL (CSS LINK):** Link your style.css file to your index.html file using the <link> tag in the <head>.
5. **PRACTICAL (CSS STYLING):** In your style.css file (not the HTML), write CSS rules to:
 - Make the <body> have a light grey background color (e.g., #f0f0f0).
 - Make your <h1> tag centered and the color red.
 - Make all <p> tags have a font-size of 18px.
 - Make all tags have a font-family of "Arial."



WEEK 9: WEB DESIGN II - CREATING SIMPLE WEBSITES

TOPIC: THE CSS BOX MODEL AND BUILDING A MULTI-PAGE SITE

INTRODUCTION: Now that we know the basics of HTML and CSS, we will learn the *most important* concept in CSS layout: The Box Model. This will allow us to control space and build a simple, coherent, multi-page website.

1: THE CSS BOX MODEL

- **THE CONCEPT:** Every single HTML element on your page is a rectangular box. The CSS Box Model describes how this box is constructed.
- **THE FOUR COMPONENTS (FROM INSIDE-OUT):**
 1. **CONTENT:** The actual text, image, or other content.
 2. **PADDING:** The transparent space *inside* the border, between the content and the border. It "pads" the content.
 3. **BORDER:** The visible line or "frame" that surrounds the padding.
 4. **MARGIN:** The transparent space *outside* the border. It "pushes" other elements away.
- **CSS EXAMPLE:**
 - `div {`
 - `width: 300px;`
 - `padding: 20px; /* 20px of space on all 4 sides, inside */`
 - `border: 2px solid black; /* A 2px black line */`
 - `margin: 40px; /* 40px of space on all 4 sides, outside */`
 - `}`

2: ID VS. CLASS (CSS SELECTORS)

- **PROBLEM:** How do you style *one* `<h1>` differently from another? You use id or class attributes in your HTML.
- **CLASS (FOR "CLASSIFICATION" - REUSABLE):**

- **HTML:** `<p class="highlight">This is important.</p>` and `<div class="highlight">This is also important.</div>`
- **CSS (USES A DOT .):**
- `.highlight {`
- `background-color: yellow;`
- `}`
- Use a class when you want to apply the *same style* to *multiple* elements.
- **ID (FOR "IDENTIFICATION" - UNIQUE):**
- **HTML:** `<div id="main-header">...</div>`
- **CSS (USES A HASH #):**
- `#main-header {`
- `border-bottom: 5px solid blue;`
- `}`
- Use an id for a *single, unique* element on a page (e.g., the main header, the footer).
An id must be unique; you cannot have two.

3: DIV VS. SPAN (STRUCTURAL ELEMENTS)

- **<DIV> (DIVISION): A BLOCK-LEVEL** container.
 - A "block" element (like `<h1>`, `<p>`) always starts on a new line and takes up the full width available.
 - A div is an empty, generic "box" used to group other elements for layout (e.g., a div with `id="container"` to wrap your whole page).
- **: AN INLINE** container.
 - An "inline" element (like `<a>`, `<img>`) sits *within* the flow of text and does not start on a new line.
 - A span is an empty, generic "inline box" used to style a *part* of a text.
 - **EXAMPLE:** `<p>This text is normal, but <span class="warning">this text</span> is red.</p>`

4: CREATING A MULTI-PAGE WEBSITE

- **PROJECT STRUCTURE:** A professional website is a folder of files that work together.
 - MyProject/ (MAIN FOLDER)
 - index.html (The homepage - this is the default file)
 - about.html
 - contact.html
 - style.css (One stylesheet for the whole site)
- **THE NAVIGATION MENU:**
 - A navigation menu is semantically a list of links. We use the `<nav>` tag to wrap an unordered list `<ul>`.
 - This *same* `<nav>` block is copied and pasted onto *every* HTML page (index.html, about.html, etc.) to ensure consistency.
 - **HTML:**
 - `<nav>`
 - `<ul>`
 - `<li><a href="index.html">Home</a></li>`
 - `<li><a href="about.html">About Us</a></li>`
 - `<li><a href="contact.html">Contact</a></li>`
 - `</ul>`
 - `</nav>`
- **LINKING PAGES (RELATIVE PATHS):**
 - Because all files are in the same folder, the href attribute is just the filename.
 - `<a href="about.html">` means "look for about.html in the same folder I am currently in."

EVALUATION

1. Draw and label the four components of the CSS Box Model, in order from outside to inside.
2. Explain the difference between padding and margin.
3. What is the difference between an id and a class attribute? Which one uses a # in CSS, and which one can be reused?

4. Explain the difference between a `<div>` (block-level) and a `<span>` (inline) element. Provide a use-case for each.
5. Why is it critical to copy the *exact same* `<nav>` HTML block onto every page of your website?

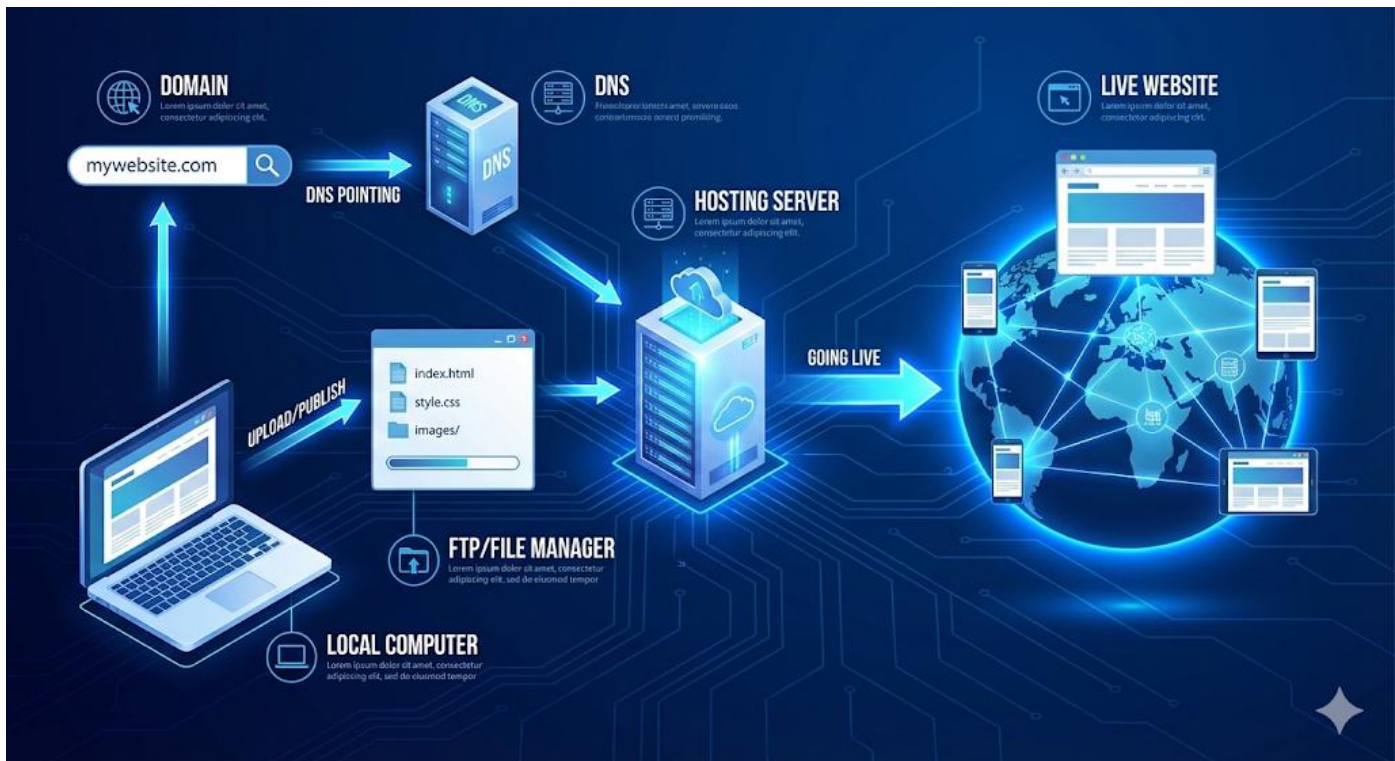
ASSIGNMENT

1. **PRACTICAL (PROJECT FOLDER):** Create a folder named MyWebsite. Inside it, create three files: index.html, about.html, and style.css.
2. **PRACTICAL (HTML):** In *both* index.html and about.html, add the basic HTML boilerplate.
3. **PRACTICAL (NAVIGATION):** In *both* index.html and about.html, create an identical `<nav>` menu that contains a `<ul>` with links to "Home" (index.html) and "About" (about.html).
4. **PRACTICAL (CSS):** In your style.css file (and make sure it is linked in the `<head>` of *both* HTML files), write CSS rules to:
 - Style your `<nav>` with a dark background color and white text.
 - Style the `<ul>` in your nav to remove the bullet points (`list-style-type: none;`).
 - Style the `<li>` in your nav to be `display: inline-block;` (this makes them horizontal).
5. **PRACTICAL (BOX MODEL):** In your index.html, create a `<div>` with a `class="main-content"`. In your style.css, give `.main-content` a width of 80%, a margin of auto (this centers it), padding of 20px, and a border of 1px solid #ccc.

WEEK 10: WEB PUBLISHING AND HOSTING

TOPIC: DOMAINS, HOSTING, AND THE PUBLISHING PROCESS

INTRODUCTION: You have built a website on your local computer. This week, we learn how to "go live" – how to make your website visible to anyone in the world on the Internet. This



involves two key purchases: a domain name and web hosting.

1: DOMAIN NAMES

- **WHAT IS A DOMAIN NAME?** A unique, human-readable "address" for a website (e.g., google.com, myschool.edu.ng).
- **RELATIONSHIP TO IP ADDRESSES:** As discussed in Week 1, web servers are found using IP addresses (e.g., 172.217.14.228). A domain name is a memorable alias that the **DNS (Domain Name System)** translates into the correct IP address.

- **DOMAIN REGISTRARS:** These are the companies accredited to sell and manage domain names (e.g., GoDaddy, Namecheap, Bluehost, or Nigerian registrars like WhoGoHost).
- **TOP-LEVEL DOMAINS (TLDS):** This is the part *after* the dot.
 - .com (Commercial)
 - .org (Organization)
 - .net (Network)
 - .ng (Country-code for Nigeria)
 - .edu.ng (A second-level domain for Nigerian educational institutions)

2: WEB HOSTING

- **WHAT IS WEB HOSTING?** If a domain name is the "address," web hosting is the "land" and "house" at that address. It is the service of *renting space* on a web server where your website's files (HTML, CSS, images) will be stored 24/7.
- **TYPES OF WEB HOSTING:**
 1. **SHARED HOSTING (CHEAPEST):** Your website is on the *same server* as hundreds of other websites. It's cheap, but performance can be affected by "noisy neighbors."
 2. **VPS (VIRTUAL PRIVATE SERVER):** You still share a physical server, but your website is given a *virtual* slice with guaranteed resources. More expensive and flexible.
 3. **DEDICATED HOSTING (MOST EXPENSIVE):** You rent an *entire* physical server for yourself. Used by large businesses with high traffic.
 4. **CLOUD HOSTING:** A modern, flexible type where your site is run on a *network* of servers. You only pay for what you use.

3: THE PUBLISHING PROCESS (FTP)

- **PROBLEM:** How do you move your index.html and style.css files from your computer to the web server you are renting?

- **SOLUTION: FTP (FILE TRANSFER PROTOCOL).** This is the standard protocol for transferring files over the Internet.
- **FTP CLIENTS:** You use a piece of software called an "FTP client" (e.g., FileZilla, Cyberduck).
- **THE PROCESS:**
 1. Your hosting provider gives you 4 pieces of information: A **Host** (server address), a **Username**, a **Password**, and a **Port** (usually 21).
 2. You enter these into your FTP client to connect.
 3. The FTP client shows a two-paned view: "Your Computer" (left) and "The Server" (right).
 4. You drag your website files from the left pane and drop them into the **public_html** folder on the right pane.
 5. Once uploaded, your website is live.

4: "FREE" PUBLISHING OPTIONS

- **STATIC SITE HOSTS:** For simple HTML/CSS/JS websites (like the ones we are building), you do not need to pay for hosting.
- **EXAMPLES:**
 - **GITHUB PAGES:** You can upload your code to a GitHub repository, and GitHub will host it for free.
 - **NETLIFY / VERCEL:** These are modern platforms that will host your static site for free, often with a simpler drag-and-drop upload process.

EVALUATION

1. Explain the "house and address" analogy for web hosting and domain names. Which is which, and why do you need both?
2. What is the role of a "Domain Registrar"?
3. Briefly differentiate between "Shared Hosting" and "Dedicated Hosting."
4. What does "FTP" stand for, and what is its primary purpose in web publishing?

5. What is the name of the specific folder on a web server where you must upload your files to make them publicly visible?

ASSIGNMENT

1. **RESEARCH (DOMAINS):** Go to a Nigerian domain registrar (like whogohost.com or qservers.net). Search for a domain name for your "MyWebsite" project. Is .com available? Is .com.ng available? What are the different prices?
2. **RESEARCH (HOSTING):** On the same website, find the "Shared Hosting" plans. Create a small table comparing their "Starter" plan and their "Business" plan. Compare at least 3 features (e.g., Disk Space, Bandwidth, Price).
3. **RESEARCH (FTP):** Download and install the FTP client "FileZilla." Open it and identify the four fields required to connect to a server.
4. **RESEARCH (STATIC HOSTING):** Go to netlify.com. Read their homepage and write a one-paragraph summary of how their "drag-and-drop" deployment works.
5. **CRITICAL THINKING:** Why do you think "Shared Hosting" is so much cheaper than "Dedicated Hosting"? What are the *risks* of using shared hosting for a large bank?

WEEK 11-13: REVISION AND EXAMINATIONS

TOPIC: CONSOLIDATION OF ALL TERM 1 CONCEPTS

INTRODUCTION: This period is dedicated to a comprehensive review of all topics covered in SS2 First Term, from Advanced Word Processing to Web Publishing.

REVISION SCOPE:

- **ADVANCED WORD:** Mail merge, macros, templates.
- **ADVANCED EXCEL:** Pivot tables, IF, VLOOKUP, SUMIF, Goal Seek, Scenario Manager.
- **ADVANCED PRESENTATION:** Slide Master, C.R.A.P. principles, multimedia, Animation Pane, interactivity.

- **WEB DESIGN:** HTML (tags, structure), CSS (selectors, box model), and Publishing (domains, hosting, FTP).

FORMAT:

- **WEEK 11:** Intensive review of Spreadsheet concepts (Weeks 3 & 4).
- **WEEK 12:** Intensive review of Web Design concepts (Weeks 8, 9, 10).
- **WEEK 13:** Final Examinations.



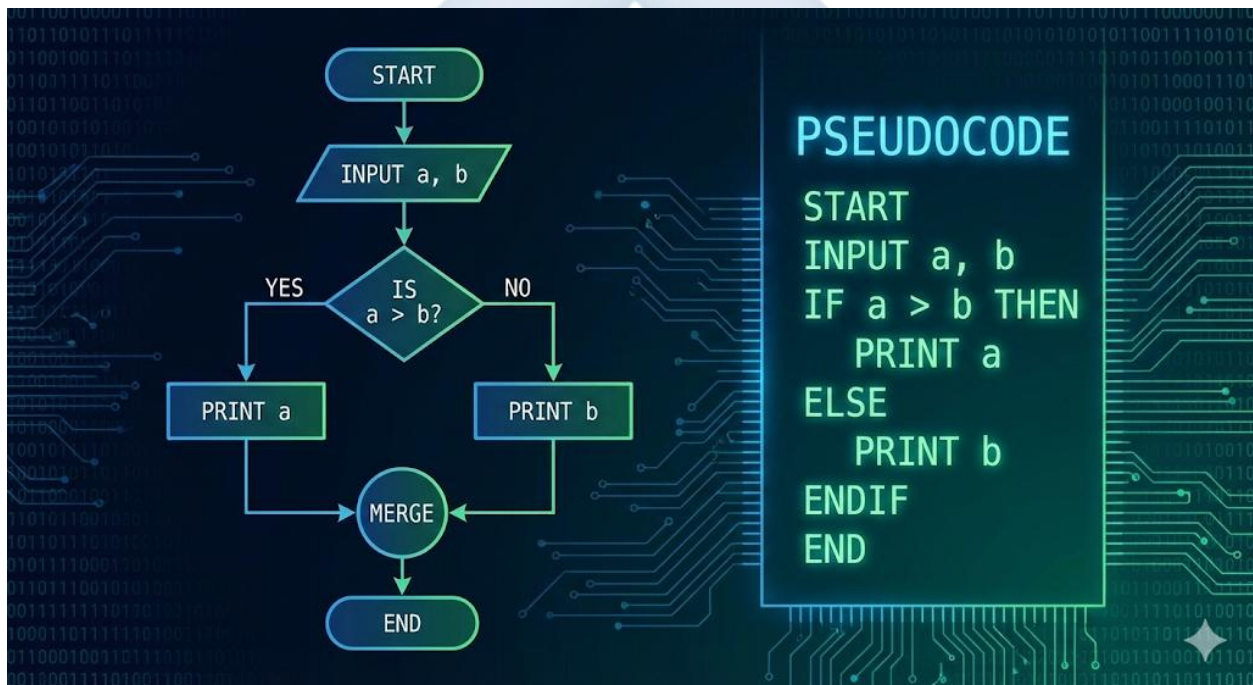
SS2 DIGITAL TECHNOLOGIES LESSON NOTES

SECOND TERM

WEEK 1: COMPUTER PROGRAMMING I

TOPIC: INTRODUCTION TO ALGORITHMS AND FLOWCHARTS

INTRODUCTION: This week, we begin the foundational study of computer programming. Before a single line of code can be written, a programmer must have a clear, logical plan. This plan is known as an algorithm. We will explore how to define these plans using plain language, pseudocode, and visual diagrams called flowcharts.



1: UNDERSTANDING ALGORITHMS

- **DEFINITION:** An algorithm is a finite, step-by-step, unambiguous set of instructions designed to solve a specific problem or perform a specific task.
- **THE "RECIPE" ANALOGY:** A good analogy is a cooking recipe. It has:
 1. **Inputs:** Ingredients (e.g., flour, eggs, sugar).
 2. **Process:** A set of instructions (e.g., "mix flour and eggs," "preheat oven").

3. **Output:** A finished product (e.g., a cake).
- **PROPERTIES OF A GOOD ALGORITHM:**
 1. **FINITE:** It must terminate after a specific number of steps. It cannot run forever.
 2. **UNAMBIGUOUS:** Each instruction must be perfectly clear, with only one possible interpretation. A computer cannot "guess" your meaning.
 3. **INPUT:** It must have zero or more well-defined inputs (the data it works with).
 4. **OUTPUT:** It must produce one or more well-defined outputs (the result).
 5. **EFFECTIVE:** Every instruction must be basic enough that it can be carried out, in principle, by a person using only paper and pencil.
 - **REPRESENTING ALGORITHMS (PSEUDOCODE):**
 - **DEFINITION:** Pseudocode (meaning "false code") is a way of writing an algorithm using plain English-like statements *before* translating it into a specific programming language.
 - **PURPOSE:** It allows the programmer to focus on the *logic* of the problem without worrying about the *syntax* (the strict grammar rules) of a language like Python or JavaScript.
 - **EXAMPLE:** Algorithm to find the average of three numbers.
 - START
 - GET number1
 - GET number2
 - GET number3
 -
 - $\text{SET total} = \text{number1} + \text{number2} + \text{number3}$
 - $\text{SET average} = \text{total} / 3$
 -
 - DISPLAY average
 - END

2: VISUALIZING LOGIC WITH FLOWCHARTS

- **DEFINITION:** A flowchart is a visual, diagrammatic representation of an algorithm. It uses a set of standard symbols connected by arrows to show the sequence of operations and the flow of control.
- **ADVANTAGES:**
 - **CLARITY:** Makes complex logic easy to understand at a glance.
 - **DEBUGGING:** Helps to trace the logic and find errors (bugs) in the plan.
 - **DOCUMENTATION:** Serves as a "blueprint" for the program, which helps in team-based projects.
- **STANDARD FLOWCHART SYMBOLS:**
 1. **TERMINATOR (OVAL/ROUNDED RECTANGLE):** Represents the START and END (or STOP) points of the algorithm.
 2. **PROCESS (RECTANGLE):** Represents an action or calculation (e.g., SET total = a + b).
 3. **INPUT/OUTPUT (PARALLELOGRAM):** Represents any action of getting data (GET number) or displaying data (DISPLAY total).
 4. **DECISION (DIAMOND):** Represents a question or a branching point. It has one input arrow and two or more output arrows (e.g., "Yes/No" or "True/False"). (e.g., Is number > 10?).
 5. **FLOW LINES (ARROWS):** Connect the symbols and show the direction of logic.
 6. **CONNECTOR (CIRCLE):** Used to connect a flowchart on the same page when lines would cross.

3: EXAMPLES OF ALGORITHMS AND FLOWCHARTS

- **EXAMPLE 1: SIMPLE PROCESS (ADDING TWO NUMBERS)**
 - **Algorithm (Pseudocode):**
 - START
 - GET num1
 - GET num2
 - SET sum = num1 + num2

- DISPLAY sum
- END
- **Flowchart:**
 - START (Terminator) -> GET num1 (Input) -> GET num2 (Input) -> SET sum = num1 + num2 (Process) -> DISPLAY sum (Output) -> END (Terminator).
- **EXAMPLE 2: A DECISION (IS A NUMBER EVEN OR ODD?)**
 - **Algorithm (Pseudocode):**
 - START
 - GET number
 -
 - SET remainder = number % 2 (The % is the Modulo operator, which finds the remainder)
 -
 - IF remainder == 0 THEN
 - DISPLAY "The number is Even"
 - ELSE
 - DISPLAY "The number is Odd"
 - END IF
 -
 - END
 - **Flowchart:**
 - START (Terminator)
 - -> GET number (Input)
 - -> remainder = number % 2 (Process)
 - -> Is remainder == 0? (Decision Diamond)
 - -> (YES branch) -> DISPLAY "Even" (Output) ->
 - -> (NO branch) -> DISPLAY "Odd" (Output) ->
 - (Both branches then point to) -> END (Terminator).

EVALUATION

1. What is an algorithm? Explain the three most important properties of a good algorithm.
2. Define "Pseudocode." What is its primary advantage for a programmer?
3. What is a flowchart? List and draw the four most common flowchart symbols and state their exact function.
4. In a flowchart, what is the key difference between the "Process" symbol and the "Decision" symbol?
5. Write the pseudocode for an algorithm that takes a student's score as input, checks if it is 50 or above, and then displays either "Pass" or "Fail".

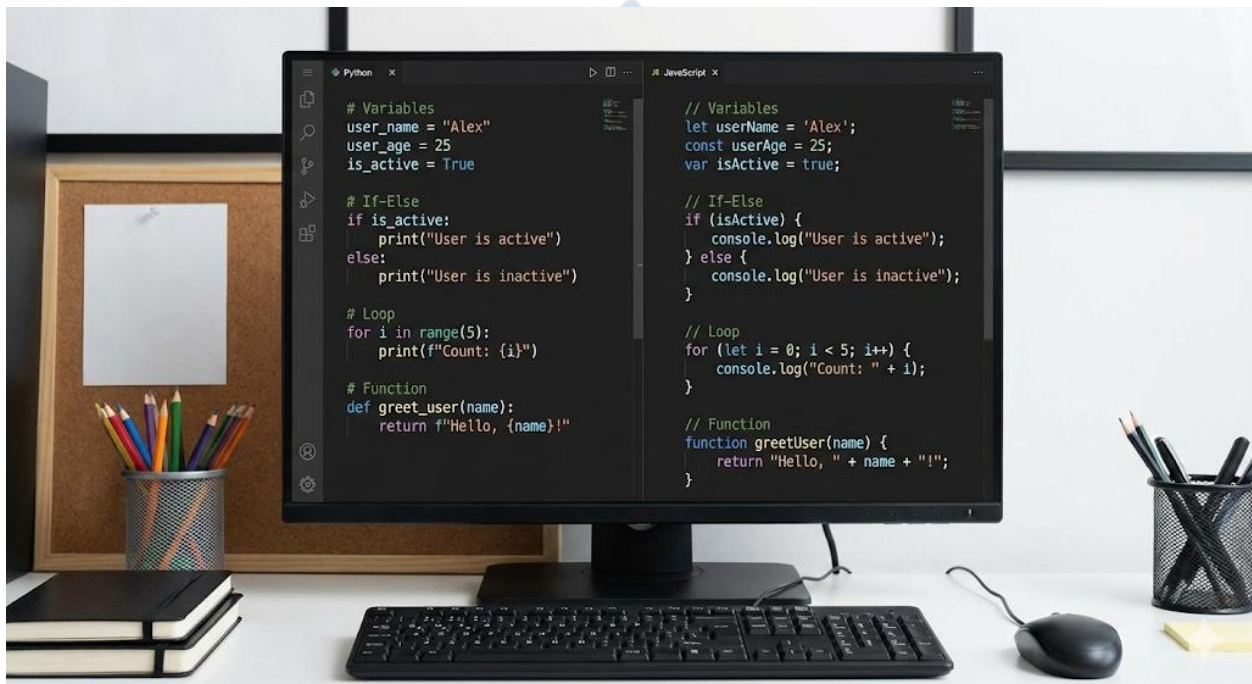
ASSIGNMENT

1. Draw a full, detailed flowchart for the "Pass/Fail" algorithm you designed in Evaluation question 5.
2. Write the pseudocode for an algorithm that calculates the area of a rectangle ($\text{Area} = \text{Length} * \text{Width}$).
3. Draw the full flowchart for the algorithm you designed in Assignment question 2.
4. Write the pseudocode for an algorithm that asks for a user's age. If the age is 18 or older, it displays "You can vote." If it is younger than 18, it displays "You are not eligible to vote."
5. Draw the full flowchart for the algorithm you designed in Assignment question 4.

WEEK 2: COMPUTER PROGRAMMING II

TOPIC: INTRODUCTION TO PYTHON AND JAVASCRIPT

INTRODUCTION: Now that we understand what an algorithm is, we must learn how to translate our logical plans into a language a computer can understand. This week, we introduce two of the most popular and powerful programming languages in the world: Python and JavaScript.



1: PROGRAMMING LANGUAGE HIERARCHY

- **LOW-LEVEL LANGUAGES:**
 - **MACHINE CODE (FIRST GENERATION):** The *only* language a CPU understands. It is a sequence of binary digits (1s and 0s). It is unreadable by humans.
 - **ASSEMBLY LANGUAGE (SECOND GENERATION):** A human-readable layer on top of machine code. It uses "mnemonics" (e.g., ADD, MOV) but is still tied directly to the hardware.
- **HIGH-LEVEL LANGUAGES (THIRD GENERATION):**

- **DEFINITION:** Languages (like Python, JavaScript, Java, C++) that use English-like syntax and mathematical notation, making them easy for humans to read, write, and understand.
- **ABSTRACTION:** They "abstract away" (hide) the complex details of the hardware (like memory management), allowing programmers to focus on solving the problem.
- **COMPILERS VS. INTERPRETERS:** High-level code must be translated into low-level machine code.
 - **COMPILER:** Reads the *entire* program (source code) at once, translates it into an executable file, and then runs it. (e.g., C++, Java).
 - **INTERPRETER:** Reads and executes the program *one line at a time*. (e.g., Python, JavaScript).
 - **IMPLICATION:** Interpreted languages are often easier to debug (you find the error on the exact line) and more flexible, but compiled languages *can* be faster.

2: INTRODUCTION TO PYTHON

- **HISTORY:** Created by Guido van Rossum and first released in 1991. Named after "Monty Python's Flying Circus."
- **PHILOSOPHY (THE "ZEN OF PYTHON"):** Emphasizes code readability, simplicity, and "batteries-included" (it comes with a huge standard library).
- **WHY IS PYTHON SO POPULAR?**
 1. **READABILITY:** Its syntax is clean, simple, and almost like writing English. It uses whitespace (indentation) to define code blocks, which forces clean layout.
 2. **VERSATILITY:** It is a "general-purpose" language. You can use it for:
 - Web Development (e.g., Django, Flask)
 - Data Science & AI (e.g., NumPy, Pandas, TensorFlow)
 - Automation & Scripting
 - Desktop Apps
 3. **HUGE COMMUNITY & ECOSYSTEM:** A massive global community means infinite tutorials, libraries, and support.
- **"HELLO, WORLD!" IN PYTHON:** This is the traditional first program.

- `# This is a comment in Python`
- `print("Hello, World!")`

3: INTRODUCTION TO JAVASCRIPT (JS)

- **HISTORY:** Created by Brendan Eich at Netscape in 1995. It was created in just 10 days.
NOTE: JavaScript is *not* related to Java. They are two completely different languages.
- **THE LANGUAGE OF THE WEB:** JavaScript is one of the three core technologies of the World Wide Web, alongside HTML (structure) and CSS (style).
- **PURPOSE:**
 1. **FRONT-END (CLIENT-SIDE):** It is the *only* language that runs natively *inside* a web browser. It is used to make websites interactive (e.g., image sliders, pop-up forms, validating a form *before* it is submitted).
 2. **BACK-END (SERVER-SIDE):** With **Node.js**, JavaScript can now also be used to write the server-side code (the same job Python does).
- **"HELLO, WORLD!" IN JAVASCRIPT:**
 - **In a Browser (Console):**
 - `// This is a single-line comment in JavaScript`
 - `/* This is a multi-line`
 - `comment */`
 - `console.log("Hello, World!");`
 - **To show an alert on a webpage:**
 - `alert("Hello, World!");`

4: KEY DIFFERENCES (PYTHON VS. JAVASCRIPT)

Feature	Python	JavaScript
Primary Use	Back-end web, Data Science, AI, Scripting	Front-end web (browsers), Back-end (Node.js)
Syntax	Simple, clear, English-like.	C-style syntax (more "curly-braced")

		{}).
Code Blocks	Uses indentation (whitespace).	Uses curly braces { }.
Typical Casing	snake_case (e.g., my_variable)	camelCase (e.g., myVariable)
Typing	Strongly Typed (less likely to auto-convert)	Weakly Typed (will auto-convert types)

EVALUATION

1. Differentiate between a High-Level Language and a Low-Level Language.
2. Explain the fundamental difference between a Compiler and an Interpreter. Which is Python?
3. What is the "philosophy" of Python, and what does it mean that Python is a "general-purpose" language?
4. What is the *primary* role of JavaScript in web development? What *other* role can it play, thanks to Node.js?
5. What is the major *syntactical* difference between how Python defines a code block and how JavaScript defines one?

ASSIGNMENT

1. Install the latest version of Python on your computer (from python.org). Open the IDLE (the built-in editor) and run the `print("Hello, World!")` command.
2. Write a simple Python program that prints your name and your school.
3. Write a 300-word essay on "Why is Python so popular for Data Science and Artificial Intelligence?"
4. Open your web browser (Chrome/Firefox), right-click, and select "Inspect." Find the "Console" tab and type `console.log("I am a JavaScript programmer!")` and hit Enter.
5. Research the difference between "Strongly Typed" and "Weakly Typed" languages. Write a paragraph explaining what this means in the context of Python vs. JavaScript.

WEEK 3: PROGRAMMING BASICS

TOPIC: VARIABLES, DATA TYPES, AND OPERATORS

INTRODUCTION: This week, we learn the fundamental building blocks of *any* programming language. A program is simply the manipulation of data. We will learn how to store data (in variables), the different types of data (data types), and how to manipulate that data (with operators). We will use Python for most examples due to its clarity.



1: VARIABLES

- **THE "BOX" ANALOGY:** A variable is a named storage location (a "box") in the computer's memory where you can store a piece of data. The "name" of the variable is the *label* you put on the box.
- **ASSIGNMENT:** The = sign is the "assignment operator." It means "put the value on the right into the box on the left."
- # In this line, 'x' is the variable name, and '10' is the value.
- `x = 10`
-

- # You can change the value
- x = 20
-
- # You can store text
- name = "Bello"
-
- # You can use variables in calculations
- y = 5
- sum = x + y # The value of 'sum' is now 25
- print(sum)

NAMING CONVENTIONS (RULES):

1. Must start with a letter or an underscore (_).
2. Cannot start with a number.
3. Can only contain letters (a-z, A-Z), numbers (0-9), and underscores.
4. Names are **case-sensitive** (age is different from Age).
5. **Best Practice (Python):** Use snake_case (all lowercase, with underscores) for variable names (e.g., first_name, total_score).

2: FUNDAMENTAL DATA TYPES

- **DEFINITION:** A data type is a classification that tells the computer how to interpret a value. It defines what *kind* of data a variable holds.
- **COMMON DATA TYPES IN PYTHON:**
 1. **int (Integer):** Whole numbers, positive or negative (e.g., 10, -5, 0).
 2. **float (Floating Point):** Numbers with a decimal point (e.g., 10.5, -3.14).
 3. **str (String):** A sequence of characters (text), enclosed in single quotes ('...') or double quotes ("..."). (e.g., "Hello", 'Nigeria').
 4. **bool (Boolean):** Represents truth values. It has only two possible values: True or False. (Note the capital T and F).
- **CHECKING TYPES:** You can use the type() function in Python to see a variable's data type.

- `x = 10`
- `y = 3.14`
- `z = "Hello"`
- `a = True`
-
- `print(type(x))` # Output: <class 'int'>
- `print(type(y))` # Output: <class 'float'>
- `print(type(z))` # Output: <class 'str'>
- `print(type(a))` # Output: <class 'bool'>
- **DYNAMIC TYPING (PYTHON/JS):** In Python and JavaScript, you don't have to *declare* the type. The interpreter figures it out automatically. This is called "dynamic typing."

3: DATA TYPE CONVERSION (CASTING)

- **DEFINITION:** Sometimes you need to convert data from one type to another. This is called "type casting."
- **THE PROBLEM:** The `input()` function in Python *always* returns a string.
- `age_str = input("Enter your age: ")` # if you type 20, `age_str` is "20"
- # The line below will cause a CRASH (TypeError)
- `# new_age = age_str + 1` # Cannot add a string "20" to an integer 1
-
- # THE SOLUTION (CASTING):
- `age_int = int(age_str)` # Convert "20" to 20
- `new_age = age_int + 1`
- `print(new_age)` # Output: 21
- **CASTING FUNCTIONS:**
 - `int()`: Converts a value to an integer.
 - `float()`: Converts a value to a float.
 - `str()`: Converts a value to a string.
 - **Example:** `print("Your new age is: " + str(new_age))` (Converts 21 back to "21" to join with a string).

4: OPERATORS

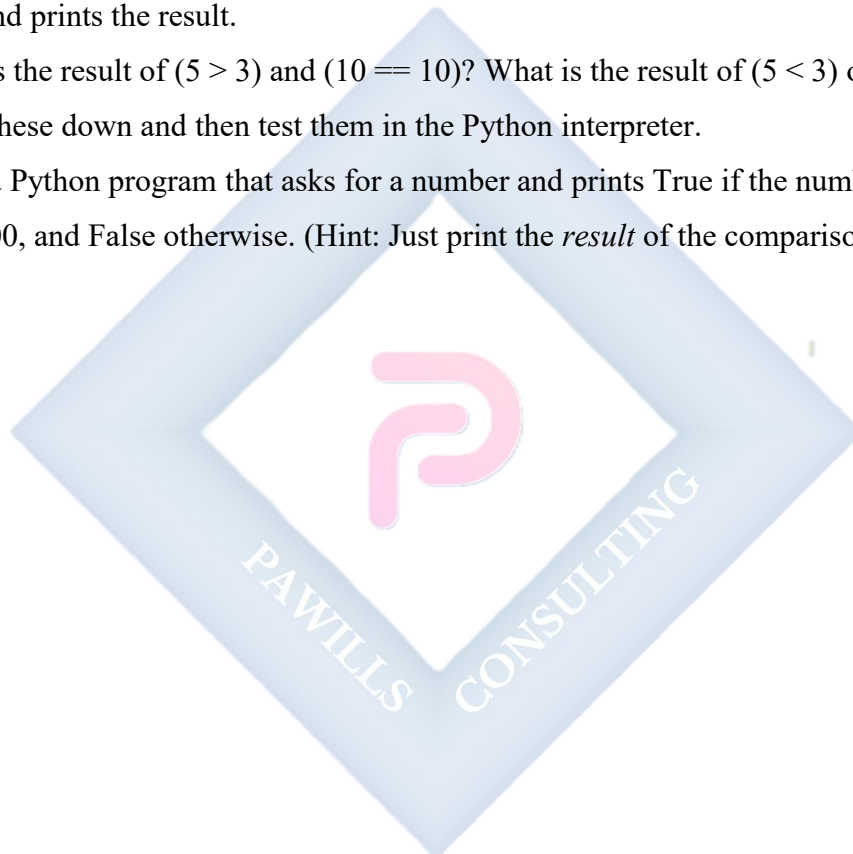
- **1. ARITHMETIC OPERATORS:** Used for mathematical calculations.
 - + (Addition), - (Subtraction), * (Multiplication), / (Division - gives a float)
 - // (Floor Division - gives an integer, no remainder)
 - % (Modulo - gives the *remainder* only)
 - ** (Exponent - "to the power of")
 - **Example:** 10 // 3 is 3. 10 % 3 is 1. 2 ** 4 is 16.
- **2. COMPARISON (RELATIONAL) OPERATORS:** Used to compare two values.
They *always* result in a Boolean (True or False).
 - == (Equal to)
 - != (Not equal to)
 - > (Greater than)
 - < (Less than)
 - >= (Greater than or equal to)
 - <= (Less than or equal to)
 - **Example:** x = 10. x > 5 is True. x == 5 is False.
- **3. LOGICAL OPERATORS:** Used to combine multiple Boolean expressions.
 - and (Returns True if *both* sides are true)
 - or (Returns True if *at least one* side is true)
 - not (Reverses the truth value)
 - **Example:** (score > 50) and (attendance > 70)

EVALUATION

1. What is a variable? Explain the "box" analogy.
2. List the four main data types in Python and give a clear example of a value for each.
3. What does it mean that Python is "dynamically typed"?
4. Why is type casting (e.g., using int()) necessary when using the input() function in Python for calculations?
5. What is the difference between the / operator and the % operator in Python?

ASSIGNMENT

1. Write a Python program that asks for your `first_name` and `last_name` in two separate variables, then prints them together.
2. Write a Python program that asks for two numbers. The program should *add*, *subtract*, *multiply*, and *divide* them, and print the result of all four operations. (Remember to use `int()` or `float()` on the input).
3. Write a Python program that asks for the length and width of a rectangle, calculates the area, and prints the result.
4. What is the result of $(5 > 3)$ and $(10 == 10)$? What is the result of $(5 < 3)$ or $(10 != 10)$? Write these down and then test them in the Python interpreter.
5. Write a Python program that asks for a number and prints `True` if the number is greater than 100, and `False` otherwise. (Hint: Just print the *result* of the comparison).

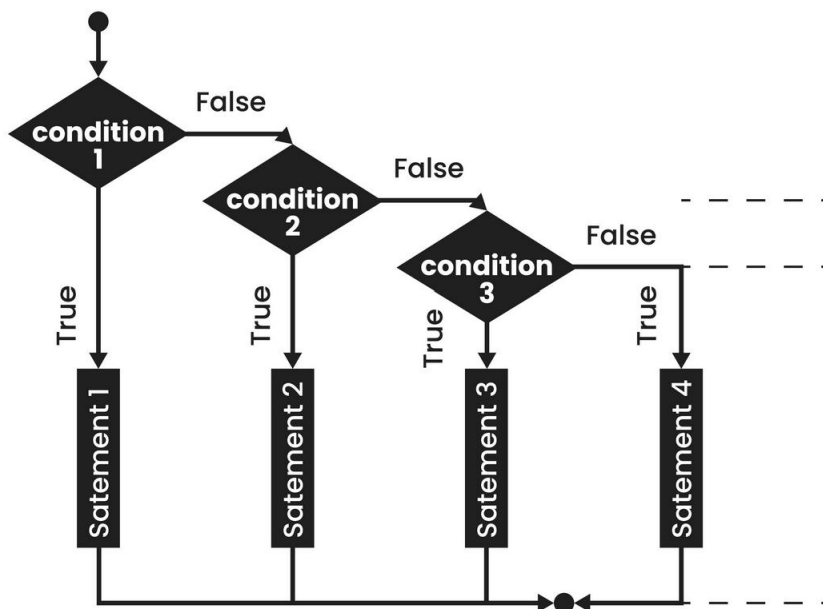


WEEK 4: CONTROL STRUCTURES

TOPIC: IF-ELSE, LOOPS, AND FUNCTIONS

INTRODUCTION: This week, we learn how to control the *flow* of a program. Until now, our programs have run from top to bottom, one line at a time. Control structures allow us to make decisions (running code only *if* a condition is met) and perform repetition (running code *over and over*), which are the keys to building complex applications.

IF-ELSE-IF STATEMENT



1: CONDITIONAL STATEMENTS (IF, ELIF, ELSE)

CONCEPT: Conditionals (or "selection") allow your program to make decisions. It executes different blocks of code based on whether a condition is True or False.

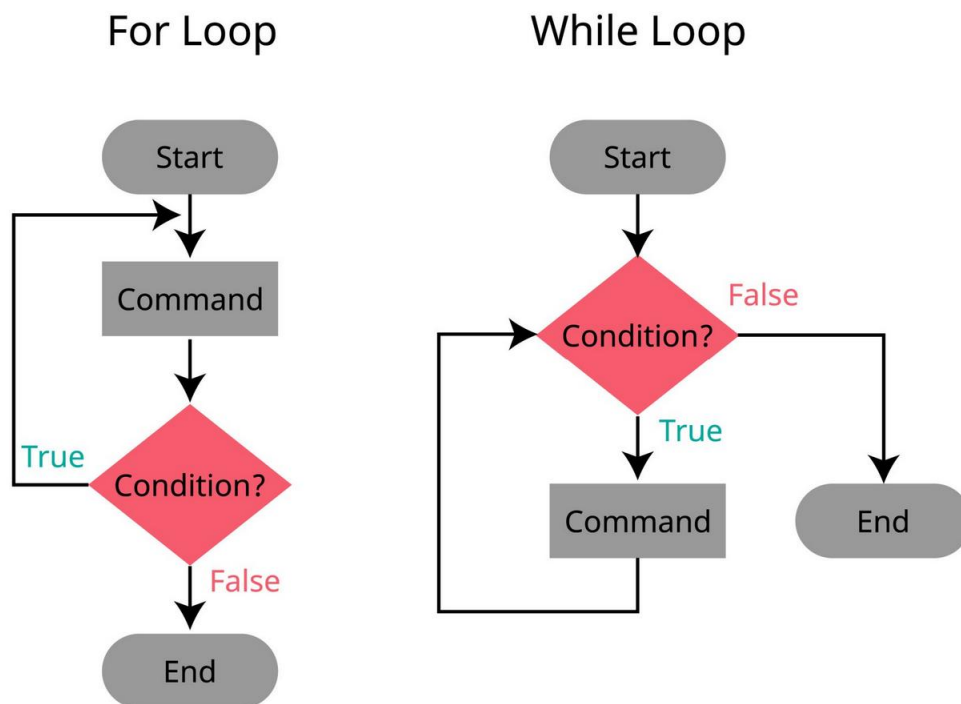
THE IF STATEMENT (THE SIMPLEST DECISION):

- age = 20

-
- `if age >= 18:`
- `print("You are eligible to vote.")`
-
- `print("Program finished.")`
- `# Note: "Program finished" always prints. The 'if' only controls the indented block.`

PYTHON'S INDENTATION: Python uses **indentation** (the space at the beginning of the line) to define a code block. This is a strict rule. JavaScript uses curly braces {}.

Getty Images



THE IF-ELSE STATEMENT (TWO CHOICES):

- `score = 45`
-
- `if score >= 50:`
- `print("You Passed.")`

- else:
- print("You Failed.")

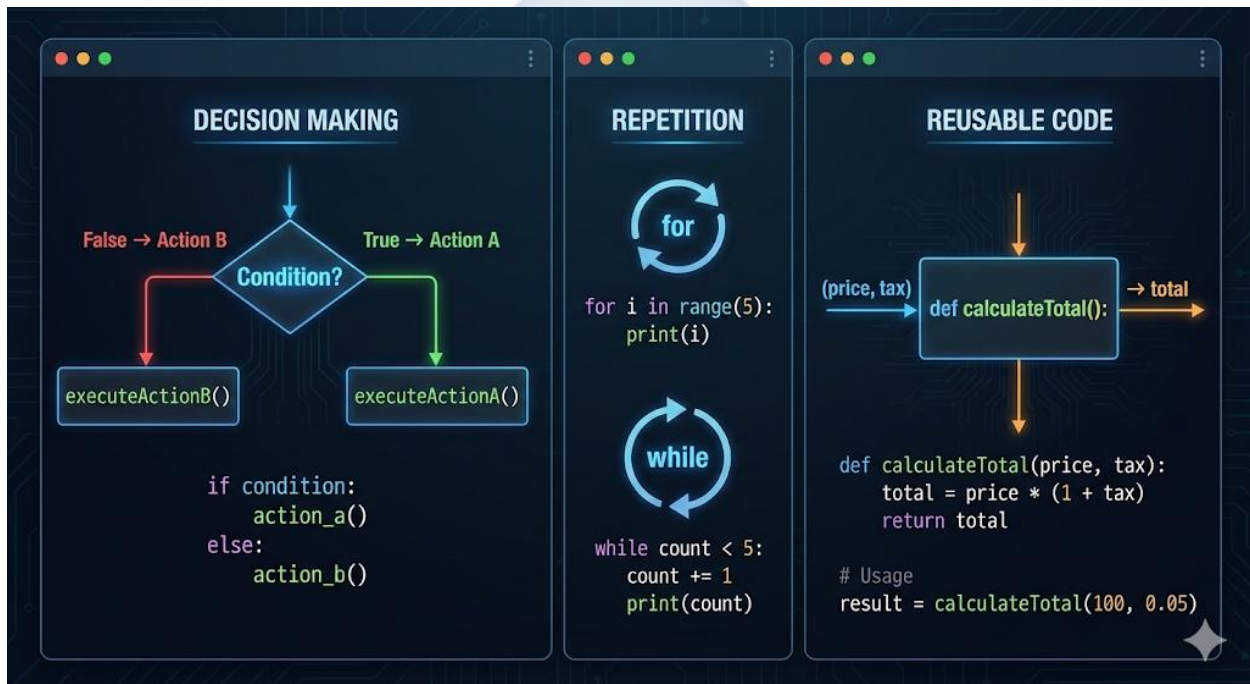
THE IF-ELIF-ELSE LADDER (MULTIPLE CHOICES): elif is short for "else if."

- score = 75
- if score >= 70:
- print("Grade: A")
- elif score >= 60:
- print("Grade: B")
- elif score >= 50:
- print("Grade: C")
- else:
- print("Grade: F")

2: REPETITION (LOOPS)

- **CONCEPT:** Loops (or "iteration") allow you to repeat a block of code multiple times, saving you from writing the same line over and over.
- **THE WHILE LOOP:**
 - **CONCEPT:** Repeats a block of code *as long as* (while) a condition remains True.
 - **STRUCTURE:** It requires a "counter" variable that is set up *before* the loop and updated *inside* the loop to avoid an "infinite loop."
 - **EXAMPLE:** Print numbers from 1 to 5.
 - count = 1 # 1. Initialize counter
 -
 - while count <= 5: # 2. Set condition
 - print(count)
 - count = count + 1 # 3. Update counter (CRITICAL!)
 -
 - print("Loop finished.")
- **THE FOR LOOP:**

- **CONCEPT:** The "counting loop." It is used to iterate over a *sequence* (like a list, or a range of numbers).
- **THE range() FUNCTION:** range(5) generates a sequence of numbers from 0 to 4 (5 numbers total).
- **EXAMPLE:** Print numbers from 0 to 4.
- for i in range(5):
- print(i)
- **EXAMPLE:** Print numbers from 1 to 5.
- for i in range(1, 6): # range(start, stop_before)
- print(i)



3: INTRODUCTION TO FUNCTIONS

CONCEPT: A function is a named, reusable block of code that performs a specific task.

WHY USE FUNCTIONS? (THE D.R.Y. PRINCIPLE):

- **D.R.Y. = Don't Repeat Yourself.**
- If you find yourself writing the same 10 lines of code in three different places, you should put those 10 lines into a function and "call" that function three times.

- **BENEFITS:** Makes code organized, readable, and much easier to update (you only have to fix a bug in one place).

DEFINING AND CALLING A FUNCTION:

- # 1. DEFINING the function (the "blueprint")
- # 'def' is the keyword to define a function
- `def greet_user():`
- `print("Hello, welcome!")`
- `print("We are glad to have you.")`
-
- # 2. CALLING the function (to "run" the code)
- `greet_user()` # Output: "Hello, welcome!" ...
- `greet_user()` # Output: "Hello, welcome!" ...

FUNCTIONS WITH PARAMETERS (INPUTS):

- `def greet_by_name(name):` # 'name' is a parameter
- `print("Hello, " + name + "! Welcome.")`
-
- # Call the function, passing an "argument" (the value)
- `greet_by_name("Aisha")` # Output: "Hello, Aisha! Welcome."
- `greet_by_name("Chinedu")` # Output: "Hello, Chinedu! Welcome."

FUNCTIONS THAT RETURN A VALUE (OUTPUTS):

- `def add_numbers(num1, num2):`
- `total = num1 + num2`
- `return total` # Sends the value back
-
- # Call the function and store its result in a variable
- `sum_result = add_numbers(5, 3)`
-

- `print(sum_result)` # Output: 8

EVALUATION

1. What is the purpose of an if-elif-else ladder?
2. What is the critical syntax rule in Python for defining a code block (e.g., inside an if or loop)?
3. Explain the difference between a while loop and a for loop. Which one is better for iterating over a known range() of numbers?
4. What is an "infinite loop," and how is it usually caused in a while loop?
5. What is a function, and what is the "D.R.Y." principle? Explain the difference between *defining* a function and *calling* it.

ASSIGNMENT

1. Write a Python program that asks for a number. If the number is positive, print "Positive." If it's negative, print "Negative." If it's zero, print "Zero." (Use if-elif-else).
2. Write a Python program that uses a while loop to print all *even* numbers from 2 to 20.
3. Write a Python program that uses a for loop to print the "5 times table" ($5 \times 1 = 5$, $5 \times 2 = 10$, ... $5 \times 12 = 60$).
4. Write (a) the def (definition) for a function called `calculate_area` that takes length and width as parameters and returns the area.
5. Write (b) the line of code to *call* your function from Q4 with a length of 10 and a width of 7, storing the result in a variable called `my_area`.

WEEK 5: MIDTERM TEST

TOPIC: ASSESSMENT OF WEEKS 1-4

INTRODUCTION: This week is dedicated to a formal assessment of all concepts covered in this term, from algorithms and flowcharts to programming basics and control structures.

SCOPE:

- **WEEK 1:** Algorithms, Pseudocode, Flowchart symbols and creation.
- **WEEK 2:** High-level vs. Low-level languages, Compilers vs. Interpreters, Python vs. JavaScript.
- **WEEK 3:** Variables, Data Types (int, float, str, bool), Type Casting, Operators (Arithmetic, Comparison, Logical).
- **WEEK 4:** Control Structures (if, elif, else), Loops (while, for), Functions (def, return, parameters).

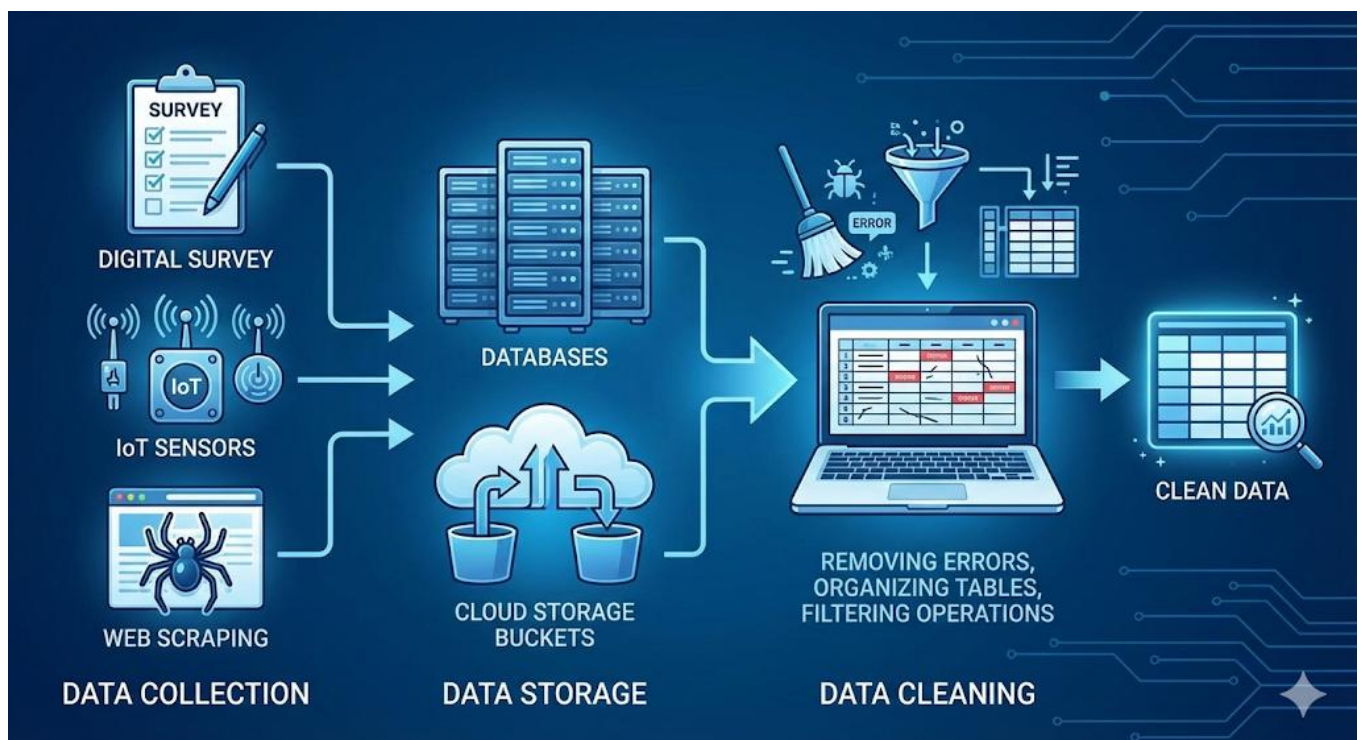
FORMAT:

- **SECTION A: MULTIPLE CHOICE QUESTIONS (20 MARKS)** - Testing definitions and concepts.
- **SECTION B: SHORT ANSWER QUESTIONS (30 MARKS)** - (e.g., "Draw the 4 main flowchart symbols," "Explain the difference between == and `=`").
- **SECTION C: PRACTICAL CODE/LOGIC (50 MARKS)** - (e.g., "Write the pseudocode for...", "Draw the flowchart for...", "Write a simple Python program to...").

WEEK 6: DATA SCIENCE BASICS

TOPIC: DATA COLLECTION, STORAGE, AND CLEANING

INTRODUCTION: We now shift from general programming to a very specific and high-demand application: Data Science. Data Science is the field of using scientific methods, algorithms, and systems to extract knowledge and insights from structured and unstructured data.



This week, we cover the first (and most important) steps in any data project.

1: WHAT IS DATA SCIENCE?

- **DEFINITION:** Data Science is an interdisciplinary field that combines:
 1. **Computer Science:** (Programming, databases, algorithms).
 2. **Statistics & Math:** (Modeling, testing, analysis).
 3. **Domain Expertise:** (e.g., knowledge of finance, medicine, or business)
- **THE GOAL:** To turn raw data into meaningful insights, predictions, and business value.

- **THE DATA SCIENCE LIFECYCLE:**
 1. Problem Definition
 2. **Data Collection**
 3. **Data Storage**
 4. **Data Cleaning**
 5. Data Analysis (Exploring)
 6. Data Modeling (e.g., Machine Learning)
 7. Data Visualization & Communication

2: DATA COLLECTION

- **DEFINITION:** The process of gathering raw data from various sources.
- **SOURCES OF DATA:**
 1. **GENERATED DATA:**
 - **Surveys/Forms:** (e.g., Google Forms, paper questionnaires).
 - **Sensors:** (e.g., weather stations, traffic cameras, smartphone GPS).
 - **Transactions:** (e.g., bank records, e-commerce purchases, call logs).
 2. **COLLECTED DATA (FROM THE WEB):**
 - **APIs (APPLICATION PROGRAMMING INTERFACES):** A clean way to "ask" a website (like Twitter) for its data in a structured format.
 - **WEB SCRAPING:** A program that automatically "reads" a website and extracts its (unstructured) HTML content, pulling out data like text or prices.
 3. **EXISTING DATA:**
 - **DATABASES:** Internal company records.
 - **PUBLIC DATASETS:** (e.g., data.gov.ng, World Bank, Kaggle).

3: DATA STORAGE

- **PROBLEM:** Once collected, data must be stored in a usable format.
- **STORAGE FORMATS:**
 1. **FLAT FILES (SIMPLE):**

- **CSV (COMMA SEPARATED VALUES):** The most common format. A simple text file that looks like a spreadsheet, with each value separated by a comma. (e.g., name,age,city\n"Bello",25,"Lagos").
- **JSON (JAVASCRIPT OBJECT NOTATION):** A format that uses key-value pairs. Very common for web APIs. (e.g., {"name": "Bello", "age": 25, "city": "Lagos"}).

2. DATABASES (COMPLEX):

- **SQL (RELATIONAL DATABASES):** (e.g., MySQL, PostgreSQL). Organizes data into highly structured *tables* with rows and columns, like multiple linked Excel sheets.
- **NOSQL (NON-RELATIONAL DATABASES):** (e.g., MongoDB). More flexible, stores data in various ways (like JSON-like documents). Used for Big Data.

4: DATA CLEANING (THE MOST IMPORTANT STEP)

- **THE "GARBAGE IN, GARBAGE OUT" (GIGO) PRINCIPLE:** Your analysis or AI model is *only* as good as your data. If your data is "garbage," your results will be "garbage."
- **FACT:** Data scientists often spend 60-80% of their time just *cleaning* and *preparing* data before they can even start analysis.
- **COMMON DATA CLEANING TASKS:**
 1. **HANDLING MISSING VALUES:** What do you do with a blank cell in a survey?
 - *Options:* Delete the entire row, or fill it in (e.g., with the average age).
 2. **REMOVING DUPLICATES:** A user submitted a form twice. You must find and remove the duplicate entry.
 3. **CORRECTING INCORRECT TYPES:** The "Age" column is a string (text). You must convert it to an int (number) for calculations.
 4. **STANDARDIZING FORMATS:**
 - One user enters "Lagos", another enters "lagos", a third enters "Lagos State". You must standardize all of them to just "Lagos".
 - Fixing "dirty" text (e.g., ?20,000 NGN needs to be cleaned to just 20000).

EVALUATION

1. Define Data Science and list the three "interdisciplinary fields" it combines.
2. What is the "GIGO" principle, and why is it so important in data science?
3. List three different *sources* of data (e.g., surveys, sensors...).
4. What is the difference between a CSV file and a JSON file?
5. List three common "data cleaning" tasks. Why do you think this step takes so much time?

ASSIGNMENT

1. **RESEARCH:** Find a "public dataset" online (e.g., from data.gov.ng or Kaggle). Write one paragraph describing the dataset, what it contains, and what kind of questions you could answer with it.
2. **RESEARCH:** What is a "web API"? In 300 words, explain what it is, using the analogy of a "waiter in a restaurant."
3. **PRACTICAL:** Create a simple CSV file in a text editor (like Notepad) for 5 students. The columns should be student_id, name, and score.
4. **CRITICAL THINKING:** You are given a large dataset of student records. The "Age" column has 30% missing values. Would you (a) delete all rows with missing ages or (b) fill them in with the average age? Explain the *advantages* and *disadvantages* of your choice.
5. **CRITICAL THINKING:** Look at this "dirty" data: ["Lagos", "Kano", "lagos ", "Port Harcourt", "KANO", "Lagos"]. Write down the "standardized" (clean) list of this data.

WEEK 7: MIDTERM BREAK



WEEK 8: BIG DATA AND ANALYTICS

TOPIC: MEANING, APPLICATIONS, AND TOOLS OF BIG DATA

INTRODUCTION: "Data" is what we handled in Week 6 (e.g., a spreadsheet). "Big Data" is a term for data that is so large, fast, and complex that it is impossible to handle with traditional tools (like a single computer or standard Excel). This week, we explore this concept and its



impact.

1: MEANING (THE 3 VS OF BIG DATA)

- **DEFINITION:** Big Data is defined by the "3 Vs" (or sometimes 5 Vs).
 1. **VOLUME (THE SIZE):** The sheer *amount* of data being generated. We are no longer talking about Megabytes (MB) or Gigabytes (GB), but Terabytes (TB), Petabytes (PB), and Exabytes (EB).

- *Example:* Every flight of a 787 jet engine generates terabytes of sensor data.
- 2. **VELOCITY (THE SPEED):** The *speed* at which new data is being created and must be processed.
 - *Example:* Stock market data, Twitter posts (thousands per second), or real-time GPS locations from millions of phones.
- 3. **VARIETY (THE COMPLEXITY):** The different *types* of data.
 - **Structured:** Clean, organized data (e.g., an SQL database, an Excel sheet).
 - **Unstructured:** "Messy" data (e.g., social media posts, emails, videos, audio files, satellite images).
 - *Challenge:* 80-90% of the world's data is unstructured.

2: APPLICATIONS OF BIG DATA ANALYTICS

- **Big Data Analytics** is the process of examining these massive datasets to uncover hidden patterns, unknown correlations, and other useful insights.
- **REAL-WORLD APPLICATIONS:**
 1. **E-COMMERCE (JUMIA, KONGA):**
 - **Recommendation Engines:** "Customers who bought this also bought..."
This is not a guess; it's calculated by analyzing the shopping habits of millions of users.
 2. **BANKING & FINANCE:**
 - **Fraud Detection:** By analyzing your "normal" spending pattern, a bank's system can instantly flag a transaction that looks abnormal (e.g., a purchase in a foreign country) and block it in real-time.
 3. **HEALTHCARE:**
 - Analyzing millions of patient records (anonymously) to find new drug interactions, predict disease outbreaks, and discover links between lifestyle and illness.
 4. **URBAN PLANNING (LAGOS STATE):**
 - Analyzing mobile phone GPS data and traffic camera feeds to understand traffic patterns and decide where to build new roads or bridges.

5. ENTERTAINMENT (NETFLIX):

- Netflix analyzes what you watch, when you pause, what you re-watch, and what you search for to decide what new movies and shows to produce.

3: TOOLS AND CONCEPTS (BRIEF)

- **THE PROBLEM:** A petabyte of data *cannot* be stored or processed on one single, massive computer. The computer would be impossibly expensive and would fail.
- **THE SOLUTION (DISTRIBUTED COMPUTING):**
 - **CONCEPT:** Instead of one *super* computer, you use a *network* (or "cluster") of hundreds or thousands of cheap, ordinary computers.
 - **STORAGE:** You "chop up" the big file and store one small piece on each computer.
 - **PROCESSING:** You give each computer a small piece of the puzzle to solve, and then combine their results at the end.
- **KEY TOOLS (TO KNOW THE NAMES):**
 - **HADOOP:** The original open-source framework for distributed storage (HDFS) and processing (MapReduce).
 - **SPARK:** A newer, faster framework that is very popular for Big Data analytics.

EVALUATION

1. What are the "3 Vs" that define Big Data? Briefly explain what each one means.
2. What is the difference between "Structured" and "Unstructured" data? Give one example of each.
3. Explain the concept of "Distributed Computing." Why is it the only practical solution for Big Data?
4. Describe a real-world application of Big Data analytics in *either* banking *or* e-commerce.
5. What is the key difference between the "data" you might analyze in Excel and "Big Data"?

ASSIGNMENT

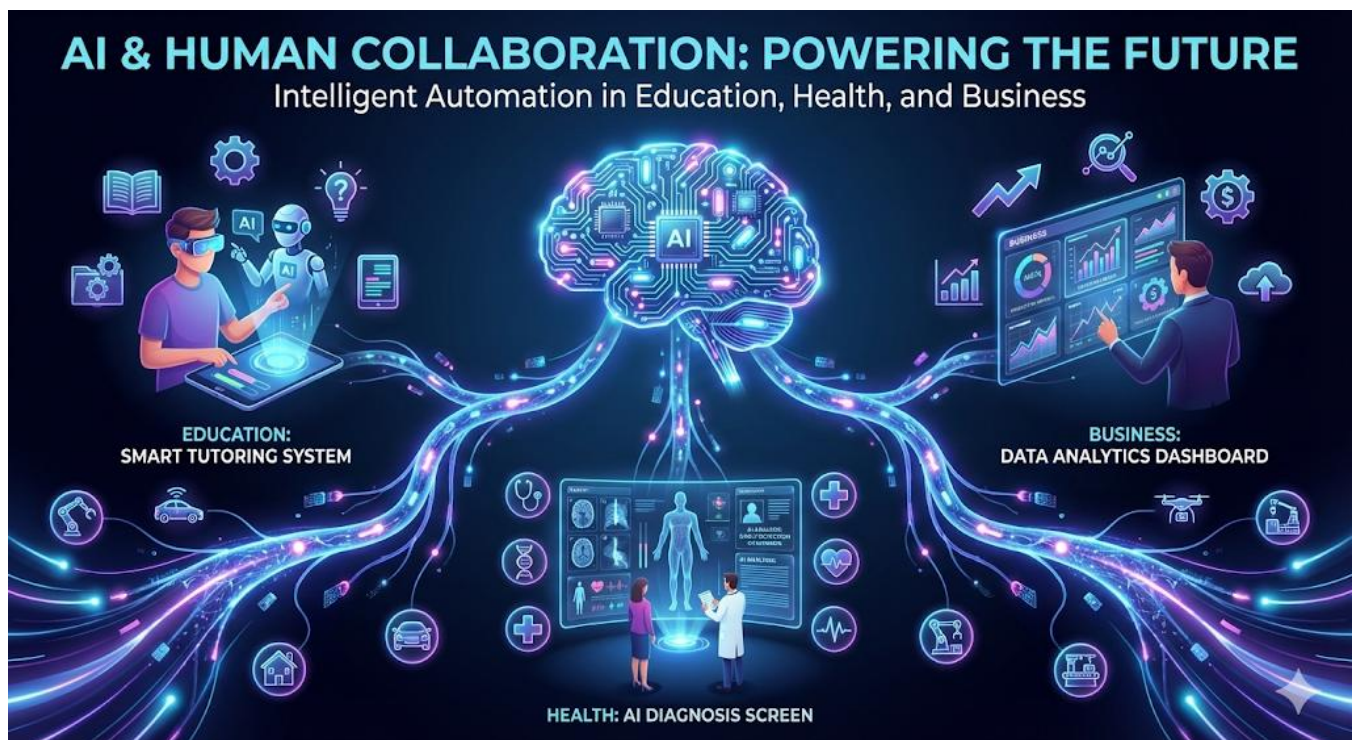
1. **RESEARCH:** Expand the "3 Vs" to the "5 Vs." What are the two additional Vs, and what do they mean?
2. **CRITICAL THINKING:** Netflix is a prime example of a company that uses Big Data. List three *specific* pieces of data you think Netflix collects about your viewing habits.
3. **CRITICAL THINKING:** How could the Lagos State government use Big Data (e.g., from phone locations and transport apps) to improve the "Danfo" bus system?
4. **RESEARCH:** What is "Hadoop"? Write a 300-word report explaining its two main components (HDFS and MapReduce) using a simple analogy (e.g., "counting all the books in a library").
5. **ETHICS:** Big Data provides great insights, but it also raises privacy concerns. Write one paragraph on what you feel is the biggest *danger* of companies collecting so much data.



WEEK 9: ARTIFICIAL INTELLIGENCE (AI)

TOPIC: APPLICATIONS IN EDUCATION, HEALTH, AND BUSINESS

INTRODUCTION: Artificial Intelligence (AI) is one of the most transformative technologies of our time. It is a broad field of computer science dedicated to creating systems that can simulate human intelligence to perform tasks like reasoning, learning, perception, and problem-



solving.

1: UNDERSTANDING AI

- **AI IS NOT A ROBOT:** A common misconception. AI is the "brain" (the software); a robot is the "body" (the hardware). A chatbot or a spam filter is AI, but it has no physical form.
- **TYPES OF AI:**

1. **WEAK AI (OR NARROW AI):** This is the *only* type of AI we have today. It is designed and trained to perform *one specific, narrow task*.
 - *Examples:* Siri/Google Assistant, a chess-playing program, facial recognition, a spam filter. (Siri cannot learn to play chess).
2. **STRONG AI (OR ARTIFICIAL GENERAL INTELLIGENCE, AGI):** This is the *hypothetical* (sci-fi) AI with the ability to understand, learn, and apply its intelligence to solve *any* problem, just like a human. **This does not exist.**

2: APPLICATIONS OF (WEAK) AI

- **AI IN EDUCATION:**

1. **PERSONALIZED LEARNING:** Apps (like Duolingo or Khan Academy) that adapt to your personal skill level. If you are struggling, it gives you easier questions. If you are advancing, it gives you harder ones.
2. **SMART TUTORS:** AI-powered chatbots that can answer student questions 24/7 or help with homework.
3. **AUTOMATED GRADING:** AI systems that can grade multiple-choice tests or even simple essays, freeing up teacher time.

- **AI IN HEALTHCARE:**

1. **MEDICAL IMAGE ANALYSIS:** AI is now *better than human radiologists* at detecting signs of diseases (like cancer or tuberculosis) in X-rays, MRIs, and CT scans.
2. **DIAGNOSIS:** AI "symptom checker" apps that can help a person identify a potential illness and tell them if they need to see a doctor.
3. **DRUG DISCOVERY:** Using AI to simulate how different chemical compounds will interact, drastically speeding up the time it takes to discover new medicines.

- **AI IN BUSINESS:**

1. **CUSTOMER SERVICE (CHATBOTS):** AI-powered bots on websites that can answer common customer questions (e.g., "What are your opening hours?" "Where is my order?").
2. **SUPPLY CHAIN OPTIMIZATION:** Using AI to predict product demand, manage inventory, and find the most efficient delivery routes for trucks.

3. **HUMAN RESOURCES (HR):** AI tools that can scan thousands of CVs in seconds to find the top 10 most qualified candidates for a job.

EVALUATION

1. What is the primary difference between AI (the "brain") and a robot (the "body")?
2. Explain the difference between "Weak AI" (Narrow) and "Strong AI" (General). Which one does *not* exist yet?
3. Give one example of Weak AI.
4. Describe one major application of AI in *healthcare*.
5. Describe one major application of AI in *business*.

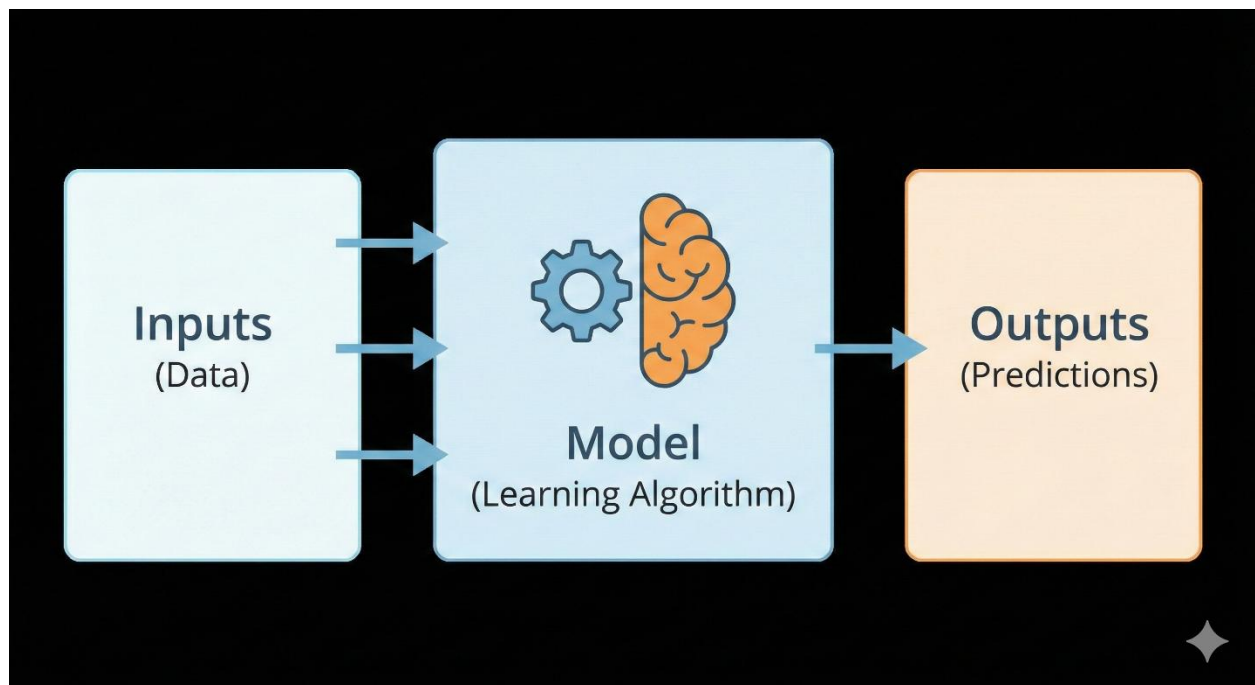
ASSIGNMENT

1. **RESEARCH:** Pick *one* of the following AI applications: (a) Self-driving cars, (b) AI-powered art generators (like Midjourney), or (c) AI in music (like Spotify's recommendation). Write 300 words on how it works and what its societal impact might be.
2. **INTERACTION:** Use an AI chatbot (like Google's Gemini, or ChatGPT). Have a conversation with it. Ask it to "explain the water cycle in the style of a pirate." Write a paragraph about your experience. Did it feel "intelligent"?
3. **ETHICS:** AI in healthcare can be very accurate. But if an AI misdiagnoses a patient, who is at fault? The doctor? The hospital? The programmer who wrote the AI? Write a 300-word opinion on this ethical problem.
4. **CRITICAL THINKING:** We discussed AI in education. What is one *advantage* and one *disadvantage* of using AI to grade student essays?
5. **CRITICAL THINKING:** AI is used to scan CVs for jobs. How could "bias" in the original data (e.g., if only men were hired in the past) lead to an AI discriminating against female candidates?

WEEK 10: MACHINE LEARNING BASICS

TOPIC: CONCEPT AND SIMPLE APPLICATIONS

INTRODUCTION: If AI is the broad goal of simulating human intelligence, **Machine Learning (ML)** is the *primary method* for achieving it. ML is a subfield of AI, and it is the technology that powers most of the modern AI applications we use every day.

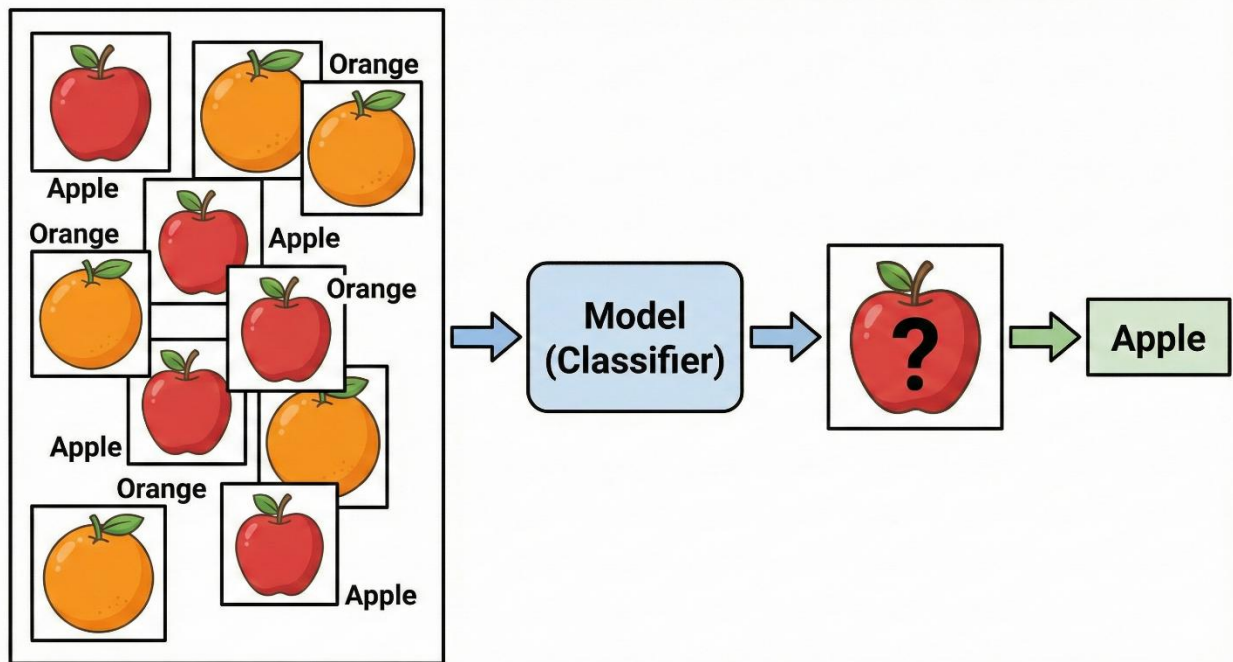


1: MACHINE LEARNING VS. TRADITIONAL PROGRAMMING

- **TRADITIONAL PROGRAMMING (WHAT WE DID IN WEEKS 1-4):**
 - We (the programmer) write *explicit rules* (like if-elif-else) to tell the computer *exactly* what to do.
 - **Input:** Data
 - **Program:** Rules
 - **Output:** Answer
 - *Problem:* What about a spam filter? You *cannot* possibly write a rule for every single "spam" word.
- **MACHINE LEARNING (THE NEW PARADIGM):**

- We (the programmer) feed the computer *data* and *examples of the answers*, and the machine *learns the rules* on its own.
- **Input:** Data + Example Answers
- **Program:** Machine Learning Algorithm
- **Output:** A "Model" (which *is* the set of rules the machine learned)

2: HOW MACHINE LEARNING WORKS (SPAM FILTER ANALOGY)



1. TRAINING:

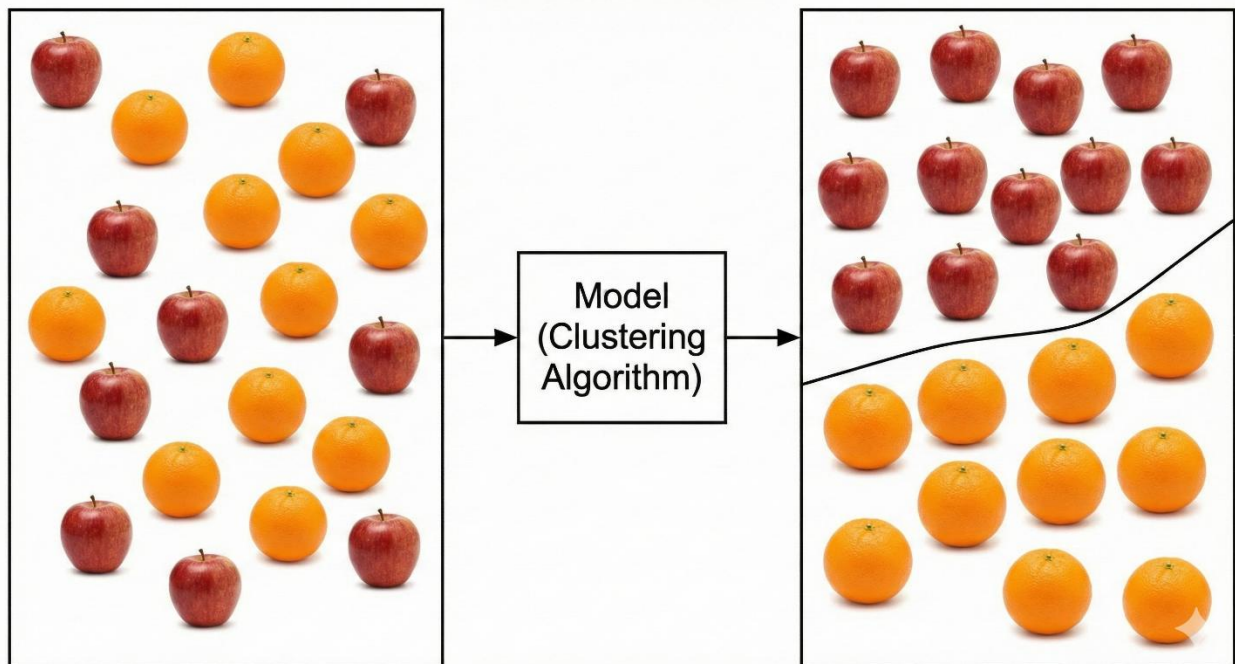
- You get a dataset of 100,000 emails.
- You *manually label* each one: "Spam" or "Not Spam" (these are the "example answers").
- You feed this labeled data into an ML algorithm.

2. LEARNING:

- The algorithm "learns" the patterns. It discovers which words (e.g., "Winner," "Free," "Viagra") are highly *correlated* with a "Spam" label.
- The output is a **Model** (a set of rules and correlations) that can predict the "spamminess" of an email.

3. PREDICTION (INFERENCE):

- A new, unseen email arrives.
- You feed it into your trained **Model**.
- The Model gives it a "spam score" (e.t., 95% likelihood of being spam) and makes a *prediction*.



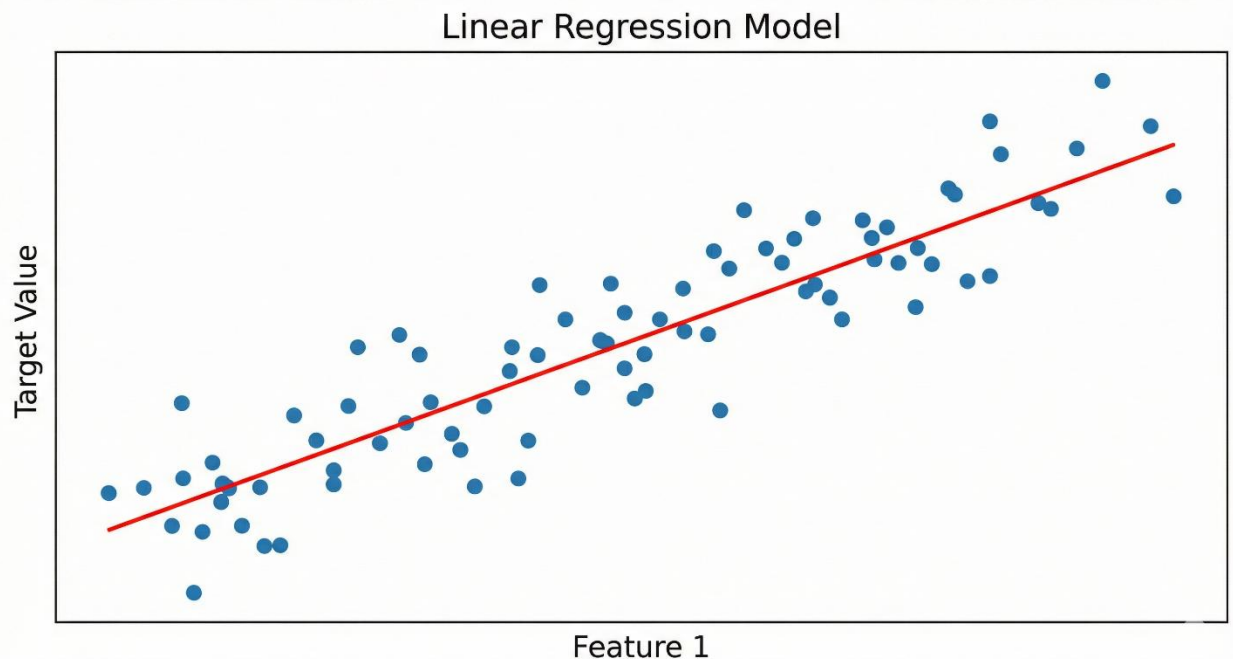
3: TYPES OF MACHINE LEARNING (SIMPLE)

1. SUPERVISED LEARNING (LEARNING WITH LABELS):

- This is the spam filter example. The data is "supervised" because it is fully labeled with the correct answers.
- **The goal:** To make predictions.
- **Examples:**
 - **Classification:** Predicting a *category* (e.g., "Spam/Not Spam," "Cat/Dog," "Fraud/Not Fraud").
 - **Regression:** Predicting a *number* (e.g., "What will be the price of this house?", "How many points will this player score?").

2. UNSUPERVISED LEARNING (LEARNING WITHOUT LABELS):

- The data has *no labels*. You don't know the "right" answer.
- **The goal:** To find hidden *patterns* or *structures* in the data.
- **Examples:**
 - **Clustering:** Automatically grouping data points into clusters. (e.g., A marketing company grouping customers into "high-spenders," "bargain-hunters," and "new-users" based on their shopping habits, *without* knowing those groups beforehand).
 - **Anomaly Detection:** Finding data points that don't fit any pattern (e.g., finding a weird sensor reading, which could be fraud or a broken machine).



4: COMMON APPLICATIONS

- **Spam Filters:** (Supervised Classification)
- **Recommendation Engines (Netflix, Jumia):** (Unsupervised Clustering, Supervised Regression)
- **Facial Recognition:** (Supervised Classification)
- **Medical Diagnosis:** (Supervised Classification)
- **Customer Segmentation:** (Unsupervised Clustering)


```
# Create a linear regression model
model = LinearRegression()

# Train the model to learn patterns from the data
model.fit(X_train, y_train)

# Now the model has learned!
print("Model trained successfully.")
```

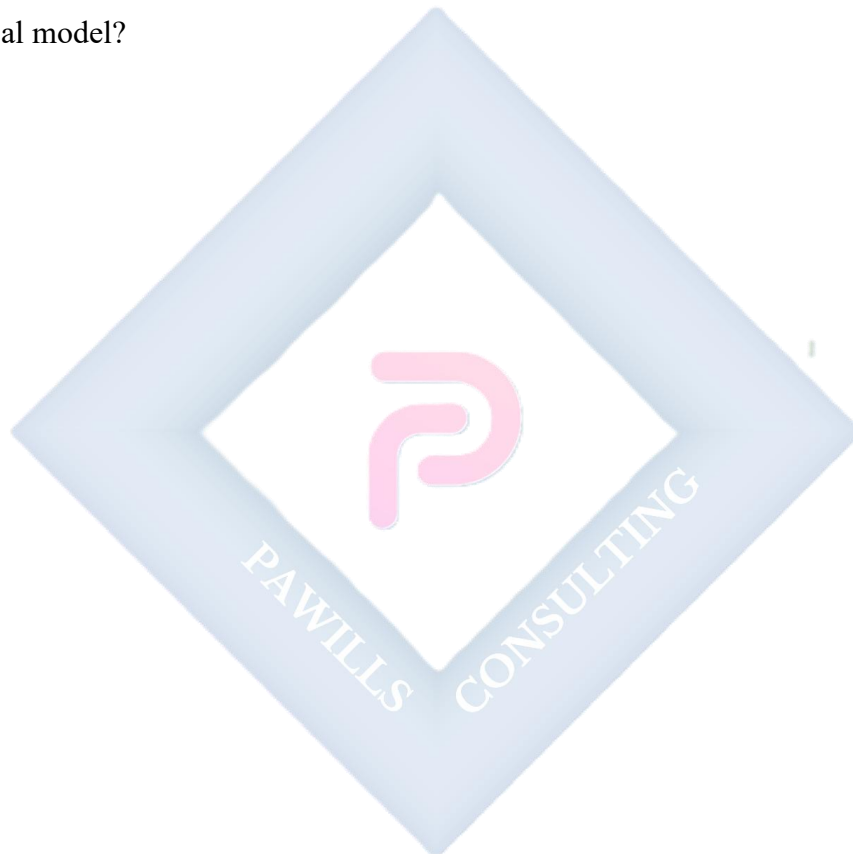
EVALUATION

1. What is the key difference between traditional programming and machine learning?
2. In machine learning, what is a "Model"? How is it "trained"?
3. What is the difference between Supervised Learning and Unsupervised Learning?
4. What *kind* of ML task is a spam filter: (a) Supervised/Classification, (b) Supervised/Regression, or (c) Unsupervised/Clustering?
5. What *kind* of ML task is "grouping customers by their secret shopping habits": (a) Supervised/Classification, (b) Supervised/Regression, or (c) Unsupervised/Clustering?

ASSIGNMENT

1. **RESEARCH:** What is "Reinforcement Learning"? How is it different from Supervised and Unsupervised learning? (Hint: think about a "reward" system, like training a dog or a game-playing AI).
2. **CRITICAL THINKING:** Jumia/Konga's "Customers who bought this also bought..." is a "recommendation engine." Is this Supervised or Unsupervised learning? (It's tricky - make an argument for either).

3. **RESEARCH:** The "k-Nearest Neighbors" (k-NN) algorithm is one of the simplest ML algorithms. Write a 300-word explanation of how it works. (Hint: "You are who your neighbors are").
4. **APPLICATION:** Think of your own school. What is *one* process or problem in your school that you think could be improved with Machine Learning? (e.g., predicting student drop-outs, creating a custom learning path, etc.).
5. **ETHICS:** A machine learning model is used to decide who gets a bank loan. It was trained on historical data from the 1980s. Why might this be a very bad, biased, and unethical model?



WEEK 11-13: REVISION AND EXAMINATIONS

TOPIC: CONSOLIDATION OF ALL TERM 2 CONCEPTS

INTRODUCTION: This period is dedicated to a comprehensive review of all topics covered in SS2 Second Term, from the fundamentals of programming logic to the applications of modern AI and Machine Learning.

REVISION SCOPE:

- **PROGRAMMING LOGIC:** Algorithms, Pseudocode, Flowcharting.
- **PROGRAMMING BASICS:** Python/JS, Variables, Data Types, Operators, if-elif-else, while/for loops, Functions.
- **DATA SCIENCE:** Collection, Storage (CSV/JSON), Cleaning (GIGO).
- **BIG DATA:** The 3 Vs (Volume, Velocity, Variety), Distributed Computing (Hadoop/Spark), Applications (Fraud, Recommendations).
- **AI/ML:** Weak vs. Strong AI, Supervised vs. Unsupervised Learning, Applications (Healthcare, Business, Spam Filters).

FORMAT:

- **WEEK 11:** Intensive review of Programming Fundamentals (Weeks 1-4).
- **WEEK 12:** Intensive review of Data Science and AI/ML (Weeks 6, 8, 9, 10).
- **WEEK 13:** Final Examinations.

SS2 DIGITAL TECHNOLOGIES LESSON NOTES

THIRD TERM

WEEK 1: MOBILE APPLICATION DEVELOPMENT I

TOPIC: INTRODUCTION TO MOBILE APPLICATIONS

INTRODUCTION: This week, we begin our study of mobile application development. A "mobile app" is a software application designed to run on a mobile device, such as a smartphone or tablet. In the modern "app economy," these applications have become the primary way businesses and users interact, surpassing traditional desktop and web usage for many tasks.



1: THE MOBILE ECOSYSTEMS (IOS VS. ANDROID)

- **THE TWO GIANTS:** The mobile world is dominated by two main ecosystems:
 1. **ANDROID (BY GOOGLE):**

- **NATURE:** Open-source. Google develops the core Android OS, but any manufacturer (Samsung, TECNO, Xiaomi) can use it and customize it.
- **MARKET SHARE:** Has the largest global market share (~70-80%) due to its availability on devices at all price points.
- **APP STORE:** Google Play Store. It has a more open and faster review process.

2. IOS (BY APPLE):

- **NATURE:** Closed-source. Apple controls *both* the hardware (iPhone, iPad) and the software (iOS).
- **MARKET SHARE:** Dominant in specific markets (like the USA) and commands the high-end, premium market.
- **APP STORE:** Apple App Store. It is known for its strict, human-led review process, which can take longer but (in theory) leads to higher-quality, safer apps.

2: TYPES OF MOBILE APPLICATIONS

• 1. NATIVE APPLICATIONS:

- **DEFINITION:** An app built *specifically* for one platform (either iOS or Android) in its native programming language.
- **TECHNOLOGY:**
 - **iOS:** Swift or Objective-C.
 - **ANDROID:** Kotlin or Java.
- **PROS:**
 - **FASTEST PERFORMANCE:** Runs "natively" on the device.
 - **BEST HARDWARE ACCESS:** Full access to the camera, GPS, accelerometer, etc.
 - **BEST USER EXPERIENCE (UI/UX):** Looks and feels exactly like the platform it was built for.
- **CONS:**
 - **EXPENSIVE:** You must build and maintain *two* separate codebases (one for iOS, one for Android).

- **SLOW DEVELOPMENT:** Takes a longer time to build two apps.
- **2. CROSS-PLATFORM (HYBRID) APPLICATIONS:**
 - **DEFINITION:** An app that is written *once* (in a shared codebase) and can be "compiled" or run on *both* iOS and Android.
 - **TECHNOLOGY:** React Native (by Facebook), Flutter (by Google), Xamarin (by Microsoft).
 - **PROS:**
 - **CHEAPER & FASTER:** "Code once, deploy everywhere." You only need one development team.
 - **GOOD PERFORMANCE:** Modern frameworks like Flutter and React Native offer near-native speed.
 - **CONS:**
 - **NOT 100% NATIVE:** May have slight performance lags or UI inconsistencies.
 - **DEPENDS ON THE FRAMEWORK:** If a new iOS or Android feature is released, the app must wait for the framework (e.g., Flutter) to support it.
- **3. PROGRESSIVE WEB APPS (PWA):**
 - **DEFINITION:** A high-quality website that is *designed* to look and feel exactly like a native app. It can be "installed" on the user's home screen directly from the browser, bypassing the app store.
 - **PROS:**
 - **NO APP STORE:** No 30% commission, no strict review process.
 - **INSTANT UPDATES:** It's a website; when you update the site, all users have the new version.
 - **CONS:**
 - **LIMITED HARDWARE ACCESS:** Cannot access all device features (though this is improving).
 - **LESS VISIBILITY:** Users must find the *website*, not the app store.

3: THE MOBILE APP DEVELOPMENT LIFECYCLE

- This is the standard process for creating a professional app:

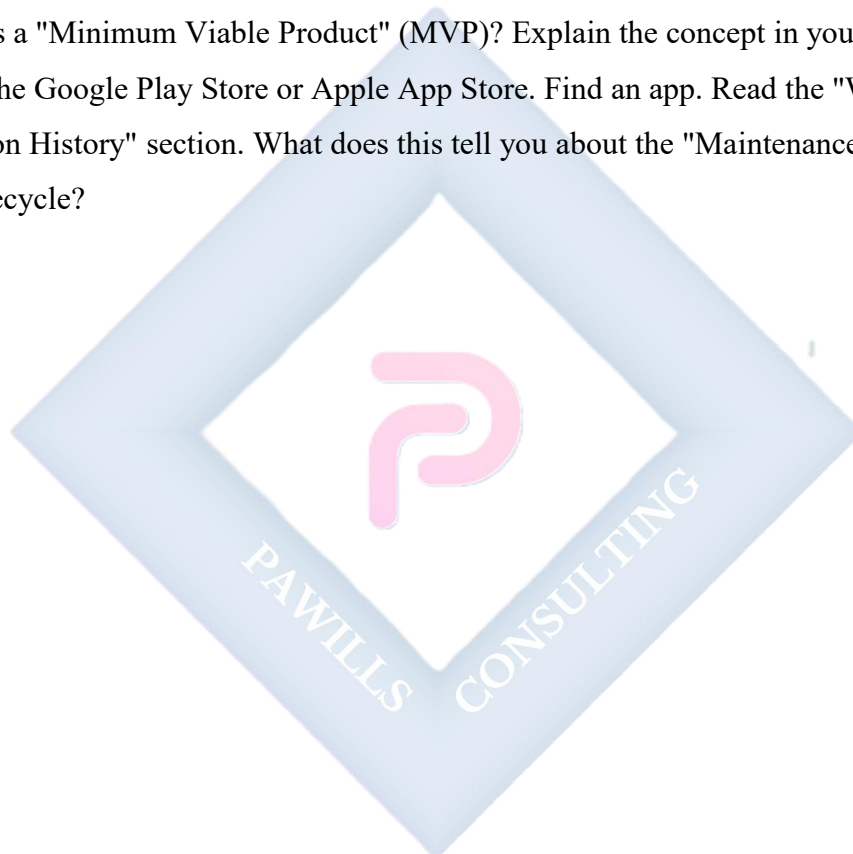
1. **STRATEGY & IDEA:** Defining the problem, the target audience, and the app's core purpose.
2. **PLANNING & RESEARCH:** Analyzing competitors, choosing the technology (Native vs. Hybrid), and defining the "Minimum Viable Product" (MVP).
3. **UI/UX DESIGN (WIREFRAMING & MOCKUPS):**
 - **Wireframe:** A basic, black-and-white "blueprint" of the app's screens.
 - **Mockup:** A full-color, high-fidelity design of what the app will look like.
4. **DEVELOPMENT (CODING):** The "back-end" (servers, databases) and "front-end" (the app itself) are built.
5. **TESTING (QUALITY ASSURANCE):** Finding and fixing bugs. Testing for performance, usability, and security.
6. **DEPLOYMENT:** Submitting the app to the Google Play Store and Apple App Store.
7. **MAINTENANCE & UPDATES:** Fixing new bugs, adding new features, and ensuring compatibility with new OS versions.

EVALUATION

1. Explain the key difference between the iOS and Android ecosystems, focusing on their "open" vs. "closed" nature.
2. What is a "Native" application? List one major advantage and one major disadvantage.
3. What is a "Cross-Platform" application? List one major advantage and one major disadvantage.
4. Why would a company choose to build a "Progressive Web App" (PWA) instead of a native app?
5. List the six main stages of the Mobile App Development Lifecycle (after the "Idea" stage).

ASSIGNMENT

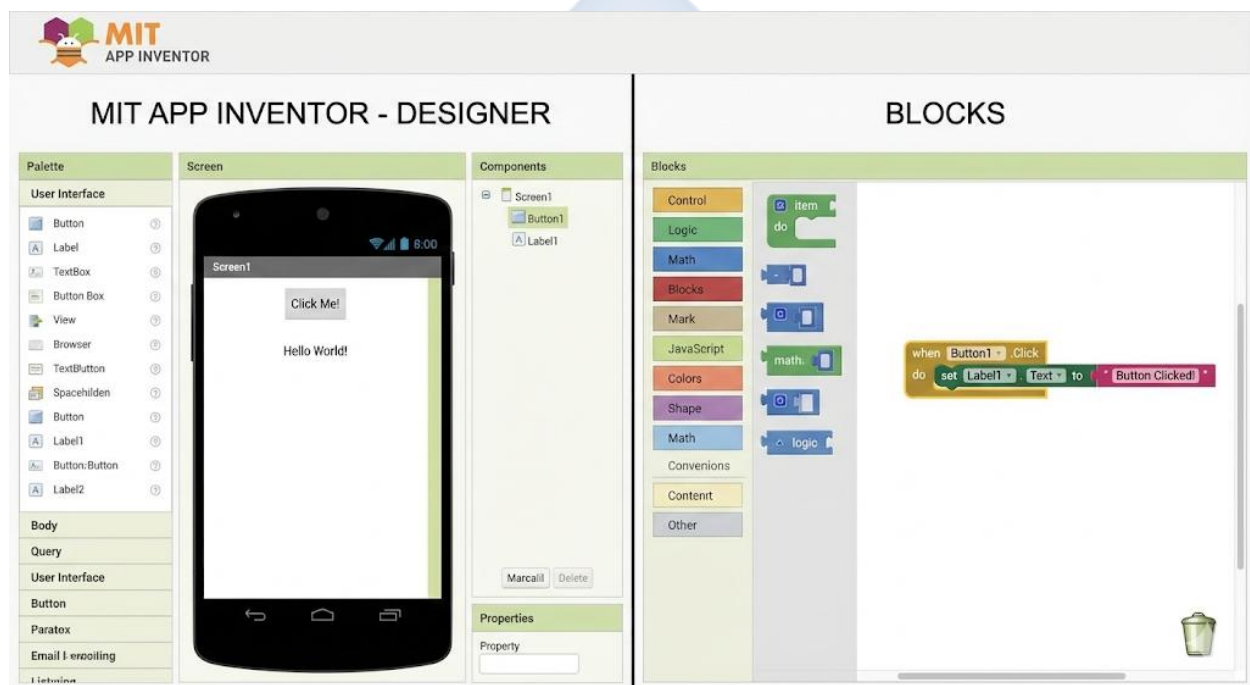
1. Pick your favorite mobile app. Do some research and determine if it is Native, Cross-Platform, or a PWA. (Hint: Look at its UI and performance). Justify your answer in one paragraph.
2. Write a 300-word essay on "If you were launching a new food delivery business in Nigeria, would you build a Native app or a Cross-Platform app first? Defend your choice."
3. What is a "Wireframe"? Draw a simple, hand-drawn wireframe for a 2-screen "School Login" app (one screen for login, one for the student's homepage).
4. What is a "Minimum Viable Product" (MVP)? Explain the concept in your own words.
5. Go to the Google Play Store or Apple App Store. Find an app. Read the "What's New" or "Version History" section. What does this tell you about the "Maintenance" phase of the app lifecycle?



WEEK 2: MOBILE APPLICATION DEVELOPMENT II

TOPIC: CREATING SIMPLE ANDROID/IOS APPS

INTRODUCTION: Last week, we learned the *theory*. This week, we explore the *practice*. Building a "real" app with native code (Kotlin/Swift) is a complex, university-level skill. However, the world of app development has been revolutionized by "No-Code/Low-Code" platforms that allow anyone—including students—to build real, working apps using visual drag-and-drop interfaces.



1: THE TWO PATHS TO BUILDING

- **1. THE "CODE-FIRST" (TRADITIONAL) PATH:**
 - **TOOLS:** Requires an "Integrated Development Environment" (IDE).
 - **Android:** Android Studio (using Kotlin/Java).
 - **iOS:** Xcode (using Swift).
 - **PROCESS:** Involves writing thousands of lines of code, managing files (XML for layouts), compiling the code, and testing on an emulator or real device.
 - **LEVEL:** Professional, complex, and time-consuming.

- **2. THE "NO-CODE / LOW-CODE" (VISUAL) PATH:**

- **DEFINITION:** These are platforms (often web-based) that replace complex programming syntax with visual, drag-and-drop blocks of logic.
- **PURPOSE:** To "democratize" development, allowing entrepreneurs, designers, and students to build functional app prototypes or even simple products.
- **EXAMPLES:** MIT App Inventor, Thunkable, Adalo, Bubble.

2: FOCUS ON: MIT APP INVENTOR

- **WHAT IS IT?** A free, web-based platform created by Google (and now managed by MIT) specifically for teaching students how to build Android apps.
- **HOW IT WORKS:** It splits development into two simple views:
 1. **THE "DESIGNER" VIEW:**
 - This is the "User Interface" (UI).
 - You have a "Palette" of components (like Button, Label, TextBox, Image, Camera, Sound).
 - You drag these components onto a "Viewer" (a screen that looks like a phone).
 - You adjust their properties (like text, color, width) in the "Properties" panel.
 2. **THE "BLOCKS" VIEW:**
 - This is the "User Logic" (the "code").
 - Instead of writing code, you snap together colorful "puzzle pieces" (blocks).
 - **Example:** A "Button.Click" block is an *event handler*. You snap a "Label.SetText" block *inside* it.

3: A "HELLO, WORLD!" APP (CONCEPTUAL DESIGN)

- This is the simplest app you can build, and it shows the core concept.
- **GOAL:** An app with a button that, when clicked, changes the text on a label.
- **1. IN THE "DESIGNER" VIEW:**

- Drag a Button component onto the screen. Set its Text property to "Click Me!".
- Drag a Label component onto the screen. Set its Text property to "Waiting...".
- **2. IN THE "BLOCKS" VIEW:**
 - From the "Button1" drawer, drag out the when Button1.Click do block.
 - From the "Label1" drawer, drag out the set Label1.Text to block.
 - From the "Text" drawer, drag out a "text string" block (the one with empty quotes).
 - Type "Hello, World!" into the text string block.
 - Snap the "text string" block into the set Label1.Text to block.
 - Snap that whole block *inside* the when Button1.Click block.
- **THE LOGIC (IN ENGLISH):** "When Button1 is clicked, set the text of Label1 to 'Hello, World!'." You have just created a functional app.

4: BEYOND THE BASICS (COMPONENTS)

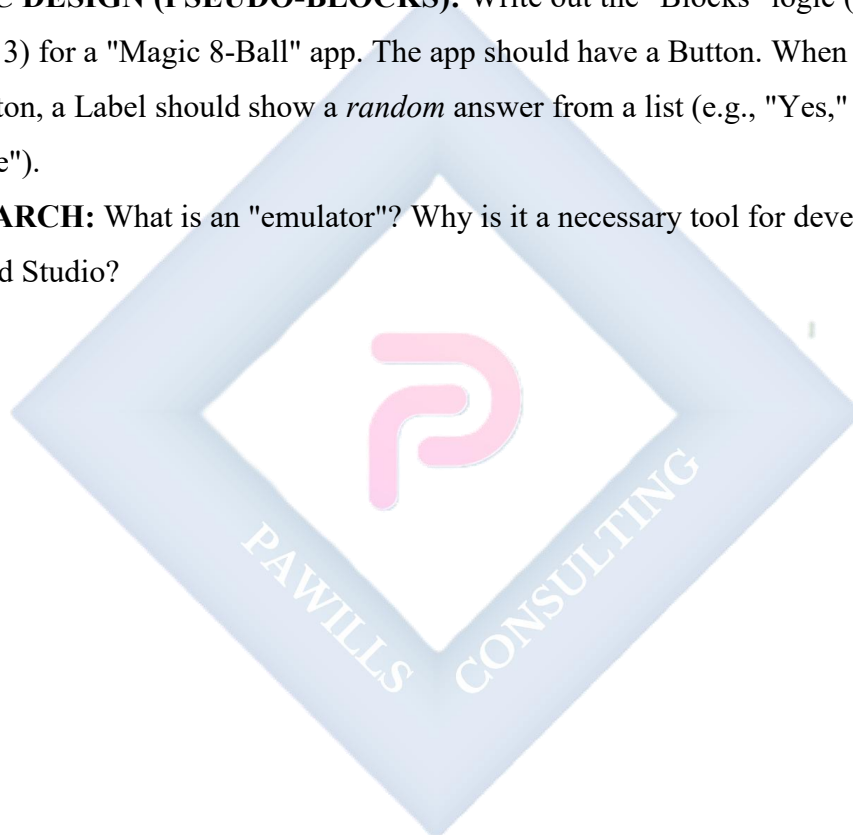
- The true power of these tools is their access to hardware components:
 - **MEDIA:** Camera (to take a picture), Sound (to play audio), TextToSpeech (to make the phone speak).
 - **SENSORS:** Accelerometer (to detect shaking/tilting), LocationSensor (to get GPS data).
 - **SOCIAL:** Sharing (to share text/images to other apps like WhatsApp).

EVALUATION

1. Explain the fundamental difference between the "Code-First" path and the "No-Code" path to app development.
2. What is "MIT App Inventor"? Who is it designed for?
3. What is the difference between the "Designer" view and the "Blocks" view in App Inventor?
4. In the "Blocks" view, what is an "event handler" block (e.g., when Button1.Click)?
5. Conceptually, what three components would you need from the "Designer" palette to build a simple "talking" app (where you type text and the phone speaks it)?

ASSIGNMENT

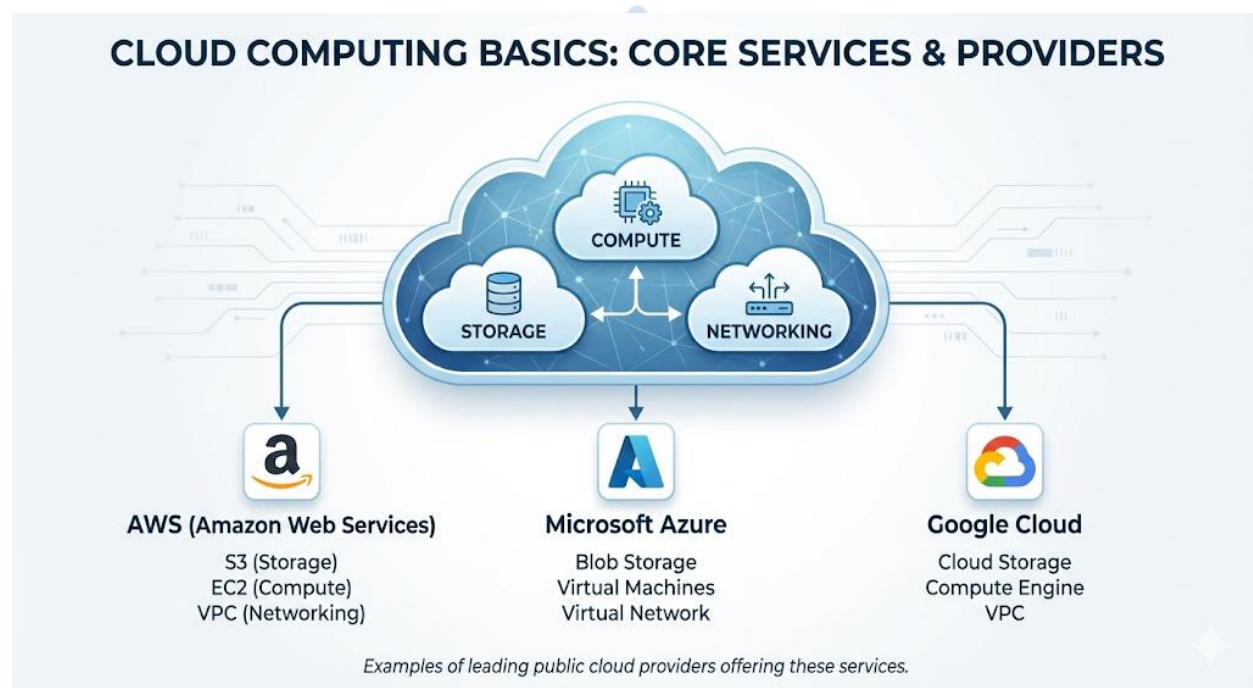
1. **RESEARCH:** Go to the website for Thunkable or Adalo. How are these platforms similar to MIT App Inventor, and how are they different? (Hint: Thunkable can build for iOS).
2. **PRACTICAL (DESIGN):** Go to appinventor.mit.edu and create a free account.
3. **PRACTICAL (BUILD):** Follow the "Hello, World!" instructions in 3 and build your first app. Test it using the "AI Companion" app on an Android phone or an emulator.
4. **LOGIC DESIGN (PSEUDO-BLOCKS):** Write out the "Blocks" logic (in plain English, like in 3) for a "Magic 8-Ball" app. The app should have a Button. When the user clicks the button, a Label should show a *random* answer from a list (e.g., "Yes," "No," "Maybe").
5. **RESEARCH:** What is an "emulator"? Why is it a necessary tool for developers who use Android Studio?



WEEK 3: CLOUD COMPUTING II

TOPIC: THE "AS-A-SERVICE" MODELS (SAAS, PAAS, IAAS)

INTRODUCTION: We have previously discussed cloud computing as "on-demand computing resources over the internet." This week, we will perform a deep dive into the *three main service models* of cloud computing. These models define the *level of control* a user has, ranging from a fully-managed service to a completely hands-on virtual datacenter.

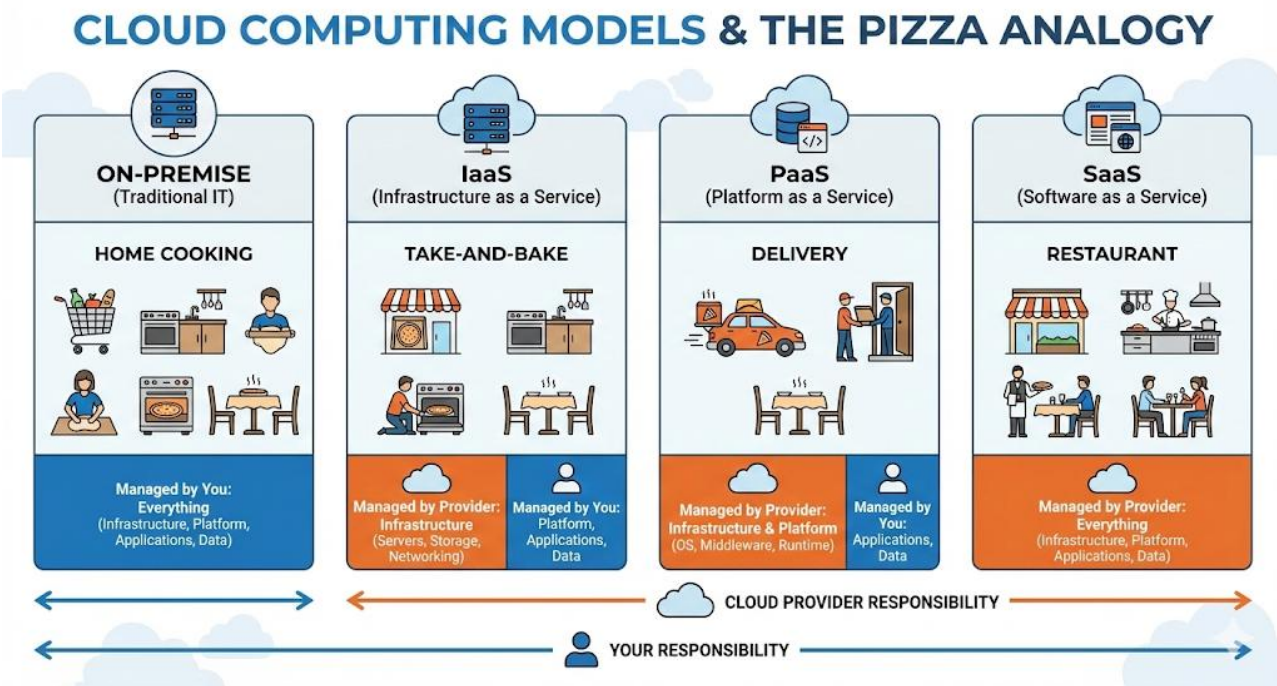


1: THE PIZZA-AS-A-SERVICE ANALOGY

- This is the most common and effective way to understand the three models:
 - **ON-PREMISE (TRADITIONAL IT): "Make Pizza at Home."**
 - You manage *everything*: the pizza dough, toppings, oven, electricity, tables, and drinks.
 - **IAAS (INFRASTRUCTURE AS A SERVICE): "Take-and-Bake Pizza."**
 - You buy a pre-made pizza from the store.
 - The vendor manages the dough, toppings, and cheese.
 - You manage the rest: your oven, electricity, tables, and drinks.

- **PAAS (PLATFORM AS A SERVICE): "Pizza Delivery."**
 - A pizza is delivered hot to your house.
 - The vendor manages *almost* everything: dough, toppings, oven, cooking.
 - You manage only the tables and drinks.
- **SAAS (SOFTWARE AS A SERVICE): "Dine at the Restaurant."**
 - You go to a restaurant and eat.
 - You manage *nothing*. The vendor handles the food, oven, tables, drinks, and cleanup. You just show up and use the service.

2: IAAS (INFRASTRUCTURE AS A SERVICE)



- **DEFINITION:** IaaS provides the fundamental building blocks of cloud IT. It is essentially *renting virtual computers and storage* in the cloud.
- **WHAT YOU RENT:** Virtual Machines (VMs), Hard Drive Storage, Networking, Firewalls.
- **YOU MANAGE:** The Operating System (e.g., Windows, Linux), the applications, the data.

- **THE VENDOR MANAGES:** The physical data centers, the physical servers, the electricity, the cooling.
- **WHO USES IT?** System administrators, IT managers, and companies that want to move their entire "server room" into the cloud to save money on hardware.
- **KEY EXAMPLES:**
 - **Amazon Web Services (AWS):** EC2 (Elastic Compute Cloud)
 - **Microsoft Azure:** Azure Virtual Machines
 - **Google Cloud:** Compute Engine

3: PAAS (PLATFORM AS A SERVICE)

- **DEFINITION:** PaaS provides a *platform* for developers to build, deploy, and manage applications *without* worrying about the underlying infrastructure. It's a "sandbox" for coders.
- **WHAT YOU RENT:** An environment to run your code, a database service, web servers.
- **YOU MANAGE:** Your application code, your data.
- **THE VENDOR MANAGES:** The OS, the hardware, the servers, the security patches, the database maintenance.
- **WHO USES IT?** Software developers. This allows a 2-person startup to build an app that can scale to millions of users, without needing a "server guy."
- **KEY EXAMPLES:**
 - **Heroku**
 - **Vercel / Netlify** (for web apps)
 - **Google App Engine**

4: SAAS (SOFTWARE AS A SERVICE)

- **DEFINITION:** SaaS provides a *completed software product* that is centrally hosted and delivered to users over the internet, almost always on a subscription basis.
- **WHAT YOU RENT:** The right to *use* the software.
- **YOU MANAGE:** Your account and the data you create.

- **THE VENDOR MANAGES:** *Everything* else: the application, the code, the servers, the OS, the maintenance, the updates.
- **WHO USES IT?** Everyone. This is the most common cloud model.
- **KEY EXAMPLES:**
 - **Google Workspace (Gmail, Google Docs)**
 - **Microsoft 365**
 - **Netflix, Spotify**
 - **Zoom**

EVALUATION

1. What are the three main service models of cloud computing?
2. Briefly explain the "Pizza-as-a-Service" analogy for all three models.
3. Which service model gives you (the user) the *most* control over the operating system?
4. Which service model gives you (the user) the *least* control?
5. Is Netflix an example of IaaS, PaaS, or SaaS? Explain why.

ASSIGNMENT

1. Your school wants to move its operations to the cloud. Which service model would you recommend for each of these needs?
 - (a) A new "Student Records" software for the front office (SaaS, PaaS, or IaaS?).
 - (b) A place for the IT administrator to run a custom-built Linux server (SaaS, PaaS, or IaaS?).
 - (c) A place for the SS3 programming students to deploy their Python web apps (SaaS, PaaS, or IaaS?).
2. Write a 300-word essay comparing the pros and cons of "On-Premise" (making pizza at home) vs. "SaaS" (eating at the restaurant).
3. Research "Google Workspace" (Gmail/Docs). Why is it a perfect example of SaaS?
4. Research "Heroku." Why is it a perfect example of PaaS for developers?
5. Research "AWS EC2." Why is it a perfect example of IaaS for system administrators?

WEEK 4: BLOCKCHAIN TECHNOLOGY

TOPIC: CONCEPT, CRYPTOCURRENCY, AND USES BEYOND FINANCE

INTRODUCTION: "Blockchain" is one of the most hyped and misunderstood technologies of the last decade. It is *not* just "Bitcoin." Bitcoin is just the *first application* of blockchain. At its core, a blockchain is a new and powerful way to store and secure data.



1: WHAT IS A BLOCKCHAIN?

- **THE "DIGITAL LEDGER" ANALOGY:**
 - Imagine a "ledger" (a record book or Excel sheet) that tracks transactions.
 - This ledger is **DISTRIBUTED**: It is *not* stored in one central place (like a bank's server). Instead, a copy of the *entire* ledger is given to thousands of computers (called "nodes") all over the world.
 - This makes it **DECENTRALIZED**: No single person, company, or government owns or controls the network.
- **THE "BLOCKS" AND THE "CHAIN":**

- When a new transaction occurs (e.g., "Adebayo pays Bisi 1 coin"), it is bundled into a "block" with other recent transactions.
- This "block" is verified by the network and then *cryptographically* "chained" to the end of the previous block, creating a **chain of blocks**.
- This cryptographic link makes the ledger **IMMUTABLE**: Once a block is added, it is practically *impossible* to change or delete it. To do so, you would have to change the block on *more than 51%* of the computers on the network at the exact same time.

2: KEY PROPERTIES OF BLOCKCHAIN

1. **DECENTRALIZATION**: No single point of failure. The network is run by its participants.
2. **IMMUTABILITY**: The data, once written, cannot be altered. This creates trust.
3. **TRANSPARENCY**: Everyone on the network can view the (anonymized) history of transactions.
4. **SECURITY**: Secured by advanced cryptography and the distributed nature of the network.

3: APPLICATION 1 - CRYPTOCURRENCY

- **DEFINITION**: A "cryptocurrency" is a digital or virtual currency that uses cryptography for security. It is the first and most famous *use-case* for blockchain.
- **BITCOIN (BTC)**: The first decentralized cryptocurrency, created in 2009 by the anonymous "Satoshi Nakamoto." Its blockchain is a public ledger that *only* records the transfer of Bitcoin from one "wallet" to another.
- **ETHEREUM (ETH)**: The second-largest cryptocurrency. Its blockchain is more advanced. It doesn't just track money; it can also run *computer programs*.

4: APPLICATIONS BEYOND FINANCE

- This is the true future of the technology.
- **1. SUPPLY CHAIN MANAGEMENT**:

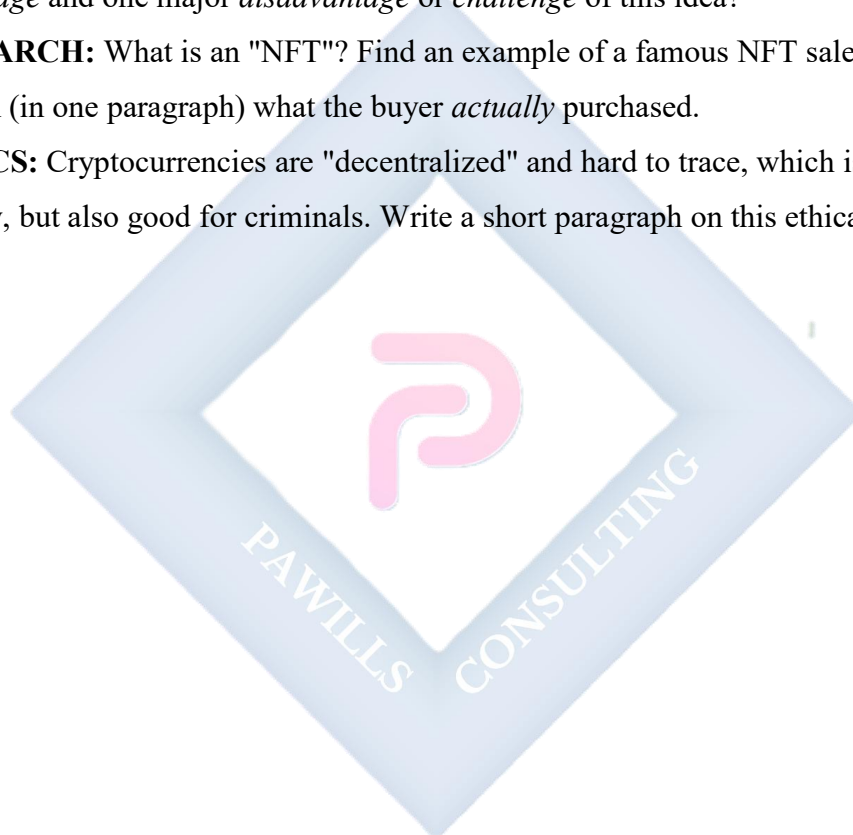
- **PROBLEM:** Where did this product come from? Is it a fake?
- **SOLUTION:** A company (e.g., a luxury brand or a food company) can use a blockchain to track a product from its creation (the farm, the factory) to the store shelf. Each step is a "block." The customer can scan a QR code to see the *entire, immutable* history of their product, proving it is authentic.
- **2. "SMART CONTRACTS" (ON ETHEREUM):**
 - **DEFINITION:** These are not legal contracts, but small *computer programs* that are stored and run on the blockchain.
 - **FUNCTION:** They automatically execute an agreement. "IF-THEN" logic.
 - **EXAMPLE:** "IF the shipping company's GPS (connected to the blockchain) reports the package is 'Delivered', THEN automatically release the payment from the buyer to the seller." This removes the need for a "middleman" to hold the money.
- **3. VOTING SYSTEMS:**
 - **PROBLEM:** Voting can be rigged, and ballots can be "lost" or changed.
 - **SOLUTION:** A blockchain-based system where each vote is a "transaction." It would be secure, anonymous, transparent (anyone could count the votes), and *immutable* (no one could change a vote after it's cast).
- **4. NON-FUNGIBLE TOKENS (NFTS):**
 - A way to prove *ownership* of a unique *digital* item (like digital art, music, or collectibles) on the blockchain.

EVALUATION

1. What is a blockchain? Explain the "digital ledger" analogy.
2. What does "Decentralized" mean in the context of a blockchain?
3. What does "Immutable" mean, and why is it an important feature of a blockchain?
4. Is "Blockchain" the same as "Bitcoin"? Explain your answer.
5. Describe one application of blockchain *other than* cryptocurrency.

ASSIGNMENT

1. **RESEARCH:** What is "Blockchain Mining" (or "Proof-of-Work")? Write a 300-word explanation of what these "miners" are doing and why they are necessary for the Bitcoin network.
2. **RESEARCH:** What is a "Smart Contract"? Write a 300-word essay on how a smart contract could be used to automatically pay a musician every time their song is streamed.
3. **CRITICAL THINKING:** We discussed using blockchain for voting. What is one major *advantage* and one major *disadvantage* or *challenge* of this idea?
4. **RESEARCH:** What is an "NFT"? Find an example of a famous NFT sale and try to explain (in one paragraph) what the buyer *actually* purchased.
5. **ETHICS:** Cryptocurrencies are "decentralized" and hard to trace, which is good for privacy, but also good for criminals. Write a short paragraph on this ethical dilemma.



WEEK 5: MIDTERM TEST

TOPIC: ASSESSMENT OF WEEKS 1-4

INTRODUCTION: This week is dedicated to a formal assessment of all concepts covered in this term, from mobile app development to blockchain technology.

SCOPE:

- **WEEK 1:** Mobile App Ecosystems (iOS/Android), App Types (Native, Cross-Platform, PWA).
- **WEEK 2:** No-Code/Low-Code Platforms, MIT App Inventor (Designer vs. Blocks), Event Handlers.
- **WEEK 3:** Cloud Service Models (IaaS, PaaS, SaaS), the "Pizza-as-a-Service" analogy, examples of each.
- **WEEK 4:** Blockchain, Decentralization, Immutability, Cryptocurrency (Bitcoin), Smart Contracts, uses beyond finance.

FORMAT:

- **SECTION A: MULTIPLE CHOICE QUESTIONS (20 MARKS)** - Testing definitions and concepts.
- **SECTION B: SHORT ANSWER QUESTIONS (30 MARKS)** - (e.g., "Explain Native vs. Hybrid," "Describe the 3 Cloud Models").
- **SECTION C: PRACTICAL/THEORY (50 MARKS)** - (e.g., "Design the 'blocks' for...", "Explain how blockchain could solve...").

WEEK 6: CYBERSECURITY III

TOPIC: ETHICAL HACKING AND PENETRATION TESTING

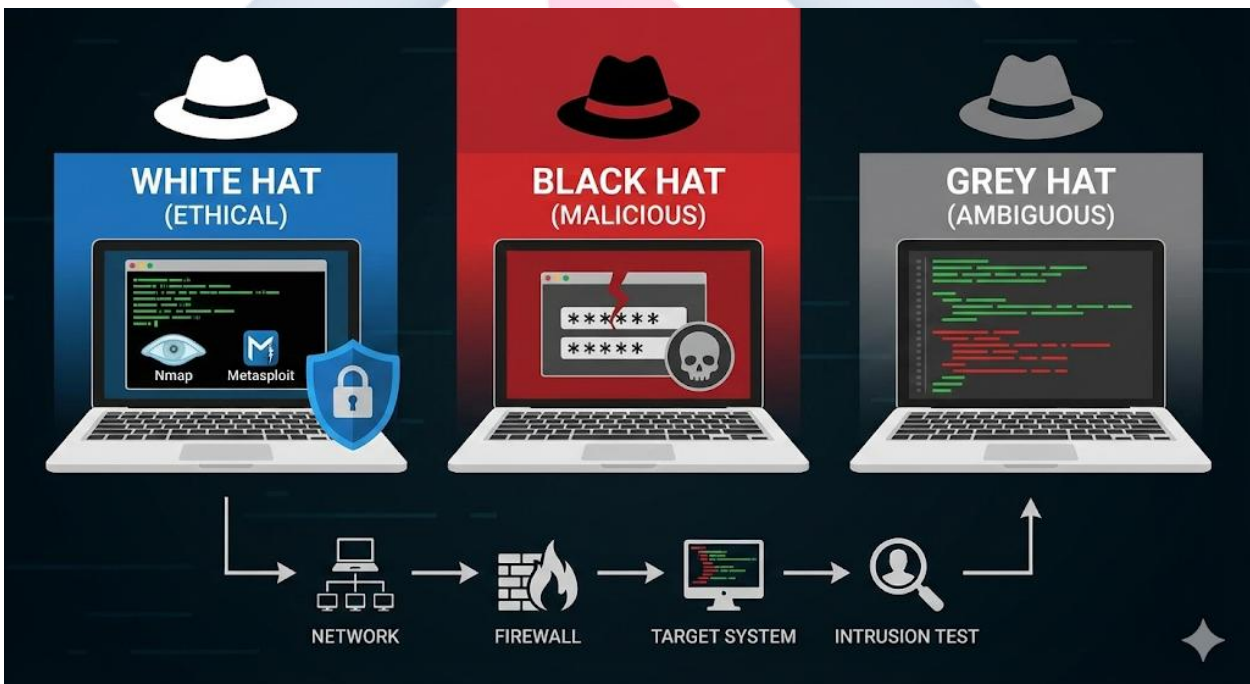
INTRODUCTION: We have previously discussed cybersecurity in terms of *defense* (passwords, firewalls). This week, we learn about *proactive defense*, also known as "offensive security." The best way to protect a system is to *think like a hacker* and find your own weaknesses before a criminal does. This is the job of an "Ethical Hacker."



1: THE "HACKER HATS" (TYPES OF HACKERS)

- **1. WHITE HAT HACKER:**
 - **THE "GOOD GUY."** Also known as an "Ethical Hacker" or "Security Researcher."
 - **MOTIVATION:** To help companies.
 - **METHOD:** Hacks with *explicit, written permission* from a company to find vulnerabilities. Their goal is to *fix* security holes.
 - **LEGALITY:** 100% legal. This is a high-paying, in-demand job.
- **2. BLACK HAT HACKER:**

- **THE "CRIMINAL."** Also known as a "Malicious Hacker."
 - **MOTIVATION:** Personal gain (money), revenge, espionage, or pure destruction.
 - **METHOD:** Hacks *without permission*. Steals data (credit cards, PII), deploys ransomware, or destroys systems.
 - **LEGALITY:** 100% illegal. This is a crime.
- **3. GREY HAT HACKER:**
 - **THE "IN-BETWEEN."**
 - **MOTIVATION:** Ambiguous. Often hacks for fun, to show off, or for "vigilante" justice.
 - **METHOD:** Hacks *without permission* (like a Black Hat), but *may* report the vulnerability to the company (like a White Hat). They might also demand a "bug bounty" (payment) to fix it, which borders on extortion.
 - **LEGALITY:** Illegal. Accessing a system without permission is a crime,



regardless of intent.

2: WHAT IS PENETRATION TESTING (PEN-TESTING)?

- **DEFINITION:** A Penetration Test (or "Pen-Test") is the *formal, authorized, and simulated* cyberattack on a computer system, network, or application to evaluate its security.
- **GOAL:** To find and exploit vulnerabilities in a safe and controlled manner, *before* a Black Hat hacker can.
- **THE DELIVERABLE:** The final "product" of a pen-test is not the "hack"; it is a **detailed report** that explains:
 1. What vulnerabilities were found.
 2. How they were exploited (step-by-step).
 3. The *risk* to the business (e.g., "We were able to steal all customer data").
 4. The *recommendations* for fixing the vulnerabilities.

3: THE 5 PHASES OF A PENETRATION TEST

1. **1. RECONNAISSANCE (INFORMATION GATHERING):**
 - "Casing the joint." The hacker gathers as much *public* information as possible about the target (e.g., employee names from LinkedIn, technologies used, IP addresses).
2. **2. SCANNING:**
 - The hacker actively "touches" the target's network.
 - Uses tools (like Nmap) to scan for open "ports" (doors), what services are running (e.g., web server, database), and what software versions they are using.
3. **3. GAINING ACCESS (EXPLOITATION):**
 - The "hack." The ethical hacker uses a known vulnerability (e.g., an unpatched server, a weak password) or a "payload" to get inside the system.
4. **4. MAINTAINING ACCESS:**
 - The hacker tries to stay in the system, "pivot" to other systems on the network, and "escalate privileges" (e.g., moving from a normal user to a "root" or "admin" user).
5. **5. COVERING TRACKS & REPORTING:**
 - A Black Hat would delete logs to hide their presence.

- A White Hat *securely documents* every step and *writes the final report* for the client.

4: COMMON ATTACK VECTORS

- **SOCIAL ENGINEERING:** Hacking the *human*. This is the *most common* attack vector.
 - **PHISHING:** A mass e-mail campaign that *looks* legitimate (e.g., from your bank) to trick you into clicking a malicious link or entering your password.
 - **SPEAR PHISHING:** A highly *targeted* phishing attack on one specific person (e.g., the CFO).
 - **PRETEXTING:** A phone call where the hacker pretends to be someone else (e.g., "Hello, this is IT, I need your password to apply a security patch").
- **TECHNICAL ATTACKS:**
 - **UNPATCHED SOFTWARE:** Running old, outdated software with known security holes. This is the *most common* technical vector.
 - **SQL INJECTION (SQLI):** An attack that targets a website's database by "injecting" malicious code into a form (like a search bar).
 - **CROSS-SITE SCRIPTING (XSS):** An attack that injects malicious code into a website, which then runs in the *victim's* browser.

EVALUATION

1. Explain the key difference between a "White Hat," "Black Hat," and "Grey Hat" hacker.
2. What is "Penetration Testing," and what is its primary goal?
3. What is the *most important* final "product" that a White Hat hacker delivers to their client?
4. List the 5 phases of a penetration test.
5. Define "Social Engineering" and give one clear example (like phishing).

ASSIGNMENT

1. **RESEARCH:** What is a "Bug Bounty" program? How do companies like Google and Apple use them to improve their security?

2. **RESEARCH:** What is "Ransomware"? How does this type of attack work, and what is the "business model" for the Black Hat hackers who use it?
3. **CRITICAL THINKING:** Why is "Reconnaissance" (Phase 1) so important? Why would a hacker spend days just "Googling" a company before attacking?
4. **RESEARCH:** What is the "Cybersecurity" or "Pen-Testing" certification called "OSCP"? What does it mean to be "OSCP certified"?
5. **ETHICS:** You are a Grey Hat. You find a major security flaw in a Nigerian bank's app that lets you see other people's account balances. What is the *ethical* (White Hat) thing to do? What is the *illegal* (Black Hat) thing to do?



WEEK 7: MIDTERM BREAK



WEEK 8: CYBER LAWS AND DIGITAL RIGHTS

TOPIC: THE NIGERIAN CYBERCRIME ACT AND GLOBAL CYBER LAWS

INTRODUCTION: As technology becomes more integrated into our lives, our laws must adapt to govern the digital space. "Cyber Law" (or "Cybercrime Law") is the field of law that deals with legal issues related to the internet and information technology. This week, we will focus on Nigeria's primary cyber law and the global concept of "Digital Rights."



1: THE NIGERIAN CYBERCRIME (PROHIBITION, PREVENTION, ETC.) ACT 2015

- **PURPOSE:** This is the primary legislation governing online activity in Nigeria. Its full title outlines its goals: "An Act to provide for the prohibition, prevention, detection, prosecution and punishment of cybercrimes in Nigeria; and for other related matters."
- **WHAT DOES IT DO?** It officially makes "cybercrime" a *crime*. Before this act, it was difficult to prosecute someone for a purely digital offense.
- **KEY PROVISIONS (SIMPLIFIED):**
 - **ILLEGAL ACCESS (HACKING):** Makes it illegal to access a computer system or network without authorization (Section 6).

- **DATA FORGERY (FRAUD):** Makes it illegal to alter or forge digital data to commit fraud (Section 7).
- **IDENTITY THEFT:** Makes it illegal to use another person's identity, email, or credentials online (Section 13).
- **PHISHING & SPAM:** Prohibits sending fraudulent messages ("phishing") and unsolicited bulk messages ("spam") (Sections 14, 15).
- **CHILD PORNOGRAPHY:** Strictly prohibits the creation, distribution, and possession of child pornography (Section 23).
- **CYBER-SQUATTING:** Makes it illegal to register a domain name (website address) that is the same as a registered trademark or brand, with the bad-faith intent to profit from it (Section 25).
- **CONTROVERSIAL PROVISION (SECTION 24):**
 - This section, titled "Cyber-stalking," is the most widely discussed.
 - It criminalizes sending a message or "causing a message to be sent" through a computer system that is:
 - (a) "grossly offensive, indecent, obscene or menacing"
 - (b) "for the purpose of... causing annoyance, inconvenience, danger, obstruction, insult, injury, criminal intimidation, enmity, hatred, ill will or needless anxiety..."
 - **THE DEBATE:** While intended to stop cyber-bullying and harassment, critics argue that terms like "annoyance" and "insult" are too vague and could be used to suppress free speech or arrest journalists and critics.

2: GLOBAL CYBER LAWS (GDPR)

- **GDPR (GENERAL DATA PROTECTION REGULATION):**
 - **WHAT IS IT?** A data privacy and security law from the *European Union (EU)*.
 - **WHY DOES IT MATTER TO NIGERIA?** It has "extra-territorial" scope. This means if *any* Nigerian company (e.g., a bank, an e-commerce site) offers services to, or processes the data of, *any person in the EU*, that Nigerian company *must* comply with the GDPR.
- **CORE PRINCIPLES OF GDPR:**

1. **"DATA SUBJECT" RIGHTS:** It gives control *back to the user*.
2. **EXPLICIT CONSENT:** Companies must ask for your permission in clear, simple language. No more pre-ticked boxes.
3. **RIGHT TO ACCESS:** You can ask any company, "What data do you have on me?" and they *must* tell you.
4. **RIGHT TO BE FORGOTTEN:** You can demand that a company *delete* your personal data.

3: DIGITAL RIGHTS

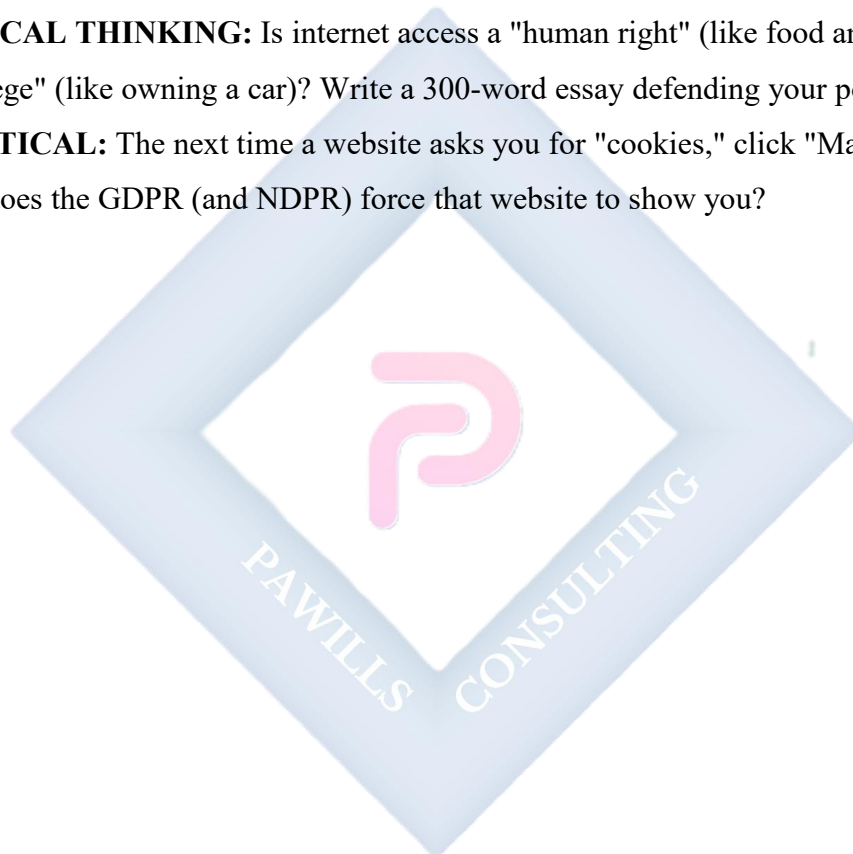
- **DEFINITION:** The concept that human rights (like freedom of speech) are not "lost" when you go online. These are the fundamental freedoms that apply to the digital world.
- **KEY DIGITAL RIGHTS:**
 1. **RIGHT TO FREEDOM OF EXPRESSION:** The right to express your opinions online, *even if* they are critical of a government or corporation. (This is often debated against "hate speech").
 2. **RIGHT TO PRIVACY:** The right to be free from unreasonable government or corporate surveillance. The right to use *encryption* to protect your communications.
 3. **RIGHT TO ACCESS:** The idea that all people should have the ability to access and use the internet. The "digital divide" (the gap between those with and without access) is a major human rights issue.

EVALUATION

1. What is the full name (or primary purpose) of the Nigerian Cybercrime Act of 2015?
2. List three specific crimes that the Cybercrime Act 2015 prohibits.
3. What is the "GDPR"? Why does a Nigerian company need to care about a European law?
4. What is the "Right to be Forgotten" under GDPR?
5. Define "Digital Rights" and give two examples (e.g., Right to Privacy).

ASSIGNMENT

1. **RESEARCH:** Find the full text of Section 24 of the Nigerian Cybercrime Act 2015. Read it. In 300 words, explain why you think it *is* or *is not* necessary to have a law against "insult" online.
2. **RESEARCH:** What is the "NDPR" (Nigeria Data Protection Regulation)? How is it similar to the European "GDPR"?
3. **CRITICAL THINKING:** Does "Freedom of Expression" online mean you should be allowed to say *anything*? Where do you think the line should be drawn between free speech and "hate speech"?
4. **CRITICAL THINKING:** Is internet access a "human right" (like food and water) or a "privilege" (like owning a car)? Write a 300-word essay defending your position.
5. **PRACTICAL:** The next time a website asks you for "cookies," click "Manage Options." What does the GDPR (and NDPR) force that website to show you?



WEEK 9: DIGITAL ENTREPRENEURSHIP

TOPIC: E-COMMERCE, FREELANCING, AND ONLINE BUSINESS

INTRODUCTION: "Entrepreneurship" is the act of creating a business to solve a problem.

"Digital Entrepreneurship" is simply using the internet and digital tools to build, run, and scale that business, often with far lower startup costs than a traditional "brick-and-mortar" company.



1: E-COMMERCE (ELECTRONIC COMMERCE)

- **DEFINITION:** The buying and selling of goods (like a phone) or services (like a Netflix subscription) over the internet.
- **KEY E-COMMERCE MODELS:**
 1. **B2C (BUSINESS-TO-CONSUMER):** The most common model. A business sells directly to an individual customer.
 - *Example:* Jumia, Konga, or a local restaurant's website.
 2. **B2B (BUSINESS-TO-BUSINESS):** A business sells its products or services to other businesses.

- *Example:* A software company that sells accounting software to law firms, or a company that sells bulk office supplies.
- 3. **C2C (CONSUMER-TO-CONSUMER):** A platform that enables individuals to sell their (often used) goods to other individuals.
 - *Example:* Jiji.ng, eBay, Facebook Marketplace.
- **THE "DROP-SHIPPING" MODEL:**
 - This is a popular *B2C* model.
 - **THE PROCESS:**
 1. You (the entrepreneur) create a beautiful website (e.g., using Shopify) and market a product (e.g., a "Smart Watch").
 2. A customer buys the watch from *your* site for N15,000.
 3. You *automatically* forward that order to your supplier (e.g., a factory in China).
 4. You pay the supplier their price (e.g., N5,000).
 5. The supplier ships the watch *directly* to the customer, with *your* brand on it.
 - **PROS:** You never have to buy or store any inventory.
 - **CONS:** You don't control shipping times or product quality.

2: FREELANCING (THE GIG ECONOMY)

- **DEFINITION:** Being self-employed and offering your *digital skills* as a service to clients on a project-by-project basis. You are your own boss.
- **WHAT SKILLS CAN YOU FREELANCE?**
 - **TECH:** Web development, app development, data analysis.
 - **CREATIVE:** Graphic design, video editing, writing, voice-over acting.
 - **ADMIN:** Virtual assistant, social media management, data entry.
- **FREELANCING PLATFORMS:** These are the "marketplaces" that connect freelancers with clients.
 - **Upwork:** A "bidding" model. Clients post a job, and freelancers bid on it.

- **Fiverr:** A "gig" model. Freelancers post their services (e.g., "I will design a logo for \$50"), and clients buy them.
- **Toptal:** An exclusive, high-end platform for the "Top 3%" of developers.
- **PROS:**
 - Flexibility (work from anywhere, set your own hours).
 - You are your own boss.
- **CONS:**
 - No stable salary (income can be inconsistent).
 - You are responsible for *everything*: finding clients, marketing, taxes, etc.

3: DIGITAL MARKETING (GETTING CUSTOMERS)

- An online business is useless if no one knows it exists. Digital Marketing is the *process of attracting customers* online.
- **KEY METHODS:**
 1. **SOCIAL MEDIA MARKETING (SMM):** Using platforms like Instagram, TikTok, and Facebook to build a brand, engage with customers, and run targeted ads.
 2. **SEARCH ENGINE OPTIMIZATION (SEO):** The "organic" (free) process of optimizing your website (e.g., with good keywords) so it appears as the *first result* on Google.
 3. **CONTENT MARKETING:** Creating *valuable* content (e.g., a blog, a "how-to" YouTube video) that attracts customers by solving their problems, rather than just running ads.

EVALUATION

1. What is E-Commerce?
2. Explain the difference between the B2C, B2B, and C2C e-commerce models, giving an example for each.
3. What is "Drop-Shipping"? What is the single biggest advantage of this model?
4. What is "Freelancing"? List three examples of digital skills that can be freelanced.

5. What is "SEO," and why is it important for an online business?

ASSIGNMENT

1. **RESEARCH:** Go to Jiji.ng. What e-commerce model is this (B2C, B2B, or C2C)? Explain your reasoning.
2. **RESEARCH:** Go to Fiverr.com. Find a service (a "gig") that *you* think you could learn to do (e.t., "Background removal from photos"). What do the top sellers charge for this?
3. **CRITICAL THINKING:** Imagine you want to start a freelance graphic design business. What would be your *first* five steps? (Hint: portfolio, platform, etc.).
4. **PRACTICAL:** You want to sell handmade jewelry online in Nigeria. Would you (a) build your own website, (b) sell on a C2C platform like Jiji, or (c) sell on social media (like Instagram)? Write 300 words justifying your choice.
5. **RESEARCH:** What is "Shopify"? How does this platform make it easy for someone to start a drop-shipping business?



WEEK 10: PROJECT WORK

TOPIC: STUDENTS DEVELOP MINI ICT PROJECTS

INTRODUCTION: This week, you will not be learning new content. Instead, you will be *applying* all the knowledge you have gained this term (and last term) by working on a mini-project. The goal is to identify a problem, design a digital solution, and build a simple "prototype" or "proof-of-concept."

1: PROJECT GOALS

- **INDIVIDUAL OR GROUP (MAX 3):** You can choose to work alone or in a small team.
- **THE GOAL:** To go through the full "development lifecycle" on a small scale:
 1. **IDENTIFY** a problem (e.g., "It's hard to track my homework").
 2. **PLAN** a solution (e.g., "A simple app to list assignments").
 3. **DESIGN** the solution (e.g., a flowchart, a UI wireframe, a database plan).
 4. **BUILD** a simple prototype.
 5. **PRESENT** your project to the class.

2: PROJECT IDEAS (CHOOSE A TRACK)

- Choose *one* of these tracks based on your interest:
- **TRACK 1: THE PROGRAMMER (PYTHON)**
 - **PROJECT:** A simple command-line application.
 - **IDEAS:**
 - A "Quiz Game" that asks 5 questions and keeps score.
 - A "Grade Calculator" that takes 5 scores and calculates the average and grade (A, B, C...).
 - A simple "Bank" app that lets you "deposit" and "withdraw" virtual money.
- **TRACK 2: THE WEB DESIGNER (HTML/CSS)**
 - **PROJECT:** A 3-page static website.
 - **IDEAS:**
 - A website for a fictional local business (e.g., a bakery, a mechanic).

- A "Tribute" website to your favorite musician or historical figure.
 - A personal "Portfolio" website for yourself.
- **TRACK 3: THE MOBILE DEVELOPER (LOW-CODE)**
 - **PROJECT:** A simple 2- or 3-screen app using **MIT App Inventor**.
 - **IDEAS:**
 - A "Soundboard" app (buttons that play different sounds).
 - A "School Info" app (buttons that link to the school's portal, show the timetable, etc.).
 - A "Magic 8-Ball" app (shake the phone, get a random answer).
- **TRACK 4: THE DATA ANALYST (EXCEL/SPREADSHEETS)**
 - **PROJECT:** Find a simple dataset, analyze it, and present your findings.
 - **IDEAS:**
 - Find a dataset on Kaggle (e.g., "Top 100 Netflix Movies").
 - Import it into Excel and create 3-4 Pivot Tables and Charts.
 - Create a 5-slide presentation on your 3 most interesting "insights."
- **TRACK 5: THE CYBERSECURITY ANALYST (RESEARCH)**
 - **PROJECT:** A detailed research report.
 - **IDEAS:**
 - A "Pen-Test Report" on a fictional company, explaining the vulnerabilities you "found" and how to fix them.
 - A deep-dive research paper on a recent, major cyberattack (e.g., a big ransomware attack) and how it happened.

3: PROJECT DELIVERABLES (WHAT YOU MUST SUBMIT)

1. **A PROJECT PLAN (1-PAGE DOCUMENT):**
 - Your name(s).
 - Which Track?
 - What is the problem you are solving?
 - What is your solution?
 - (If applicable) A simple flowchart or wireframe of your design.
2. **THE PROJECT ITSELF:**

- The .py file(s).
- The .html and .css files.
- The .aia (App Inventor) file.
- The Excel file and PowerPoint presentation.
- The Word document (report).

3. A 5-MINUTE CLASS PRESENTATION:

- Explain your project, demonstrate it if possible, and talk about what you learned.

EVALUATION

1. What is a "prototype" in the context of a project?
2. What is the purpose of a "Project Plan"?
3. Describe the "Web Designer" project track.
4. Describe the "Mobile Developer" project track.
5. What are the three "deliverables" required for this project?

ASSIGNMENT

1. Decide if you will work individually or in a group.
2. Choose your "Project Track" from 2.
3. Choose your specific "Project Idea."
4. Write your 1-page "Project Plan" and submit it for approval.
5. Begin working on your project.

WEEK 11-13: REVISION AND EXAMINATIONS

TOPIC: CONSOLIDATION OF ALL TERM 2 & 3 CONCEPTS

INTRODUCTION: This period is dedicated to a comprehensive review of all topics covered in SS2, with a focus on the advanced topics from this term. We will consolidate our knowledge and prepare for the final examinations.

REVISION SCOPE (THIS TERM):

- **Mobile Apps:** Native vs. Cross-Platform, Low-Code (App Inventor).
- **Cloud Computing:** IaaS, PaaS, SaaS, and the Pizza Analogy.
- **Blockchain:** Decentralization, Immutability, Crypto, Smart Contracts.
- **Cybersecurity:** White/Black Hat, Pen-Testing, Phishing.
- **Cyber Law:** Nigerian Cybercrime Act (2015), GDPR, Digital Rights.
- **Entrepreneurship:** E-commerce (B2C/C2C), Freelancing, SEO.

FORMAT:

- **WEEK 11:** Intensive review of Mobile, Cloud, and Blockchain (Weeks 1-4).
- **WEEK 12:** Intensive review of Cybersecurity, Cyber Law, and Entrepreneurship (Weeks 6, 8, 9). Final Project Presentations.
- **WEEK 13:** Final Examinations.